



INHA UNIVERSITY TASHKENT

SPRING SEMESTER 2024

SOC 2040

SYSTEM PROGRAMMING

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR

DR. A. R. NASEER

14 May 2024

HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

SOC 2040

SYSTEM PROGRAMMING

WEEK 14 LECTURE2

VIRTUAL MEMORY

VIRTUAL MEMORY CONCEPTS

Lecture Outline

- ❑ Virtual Memory (VM)– Basic Concepts
- ❑ Address spaces
- ❑ VM as a tool for caching
- ❑ VM as a tool for memory management
- ❑ VM as a tool for memory protection
- ❑ Address translation

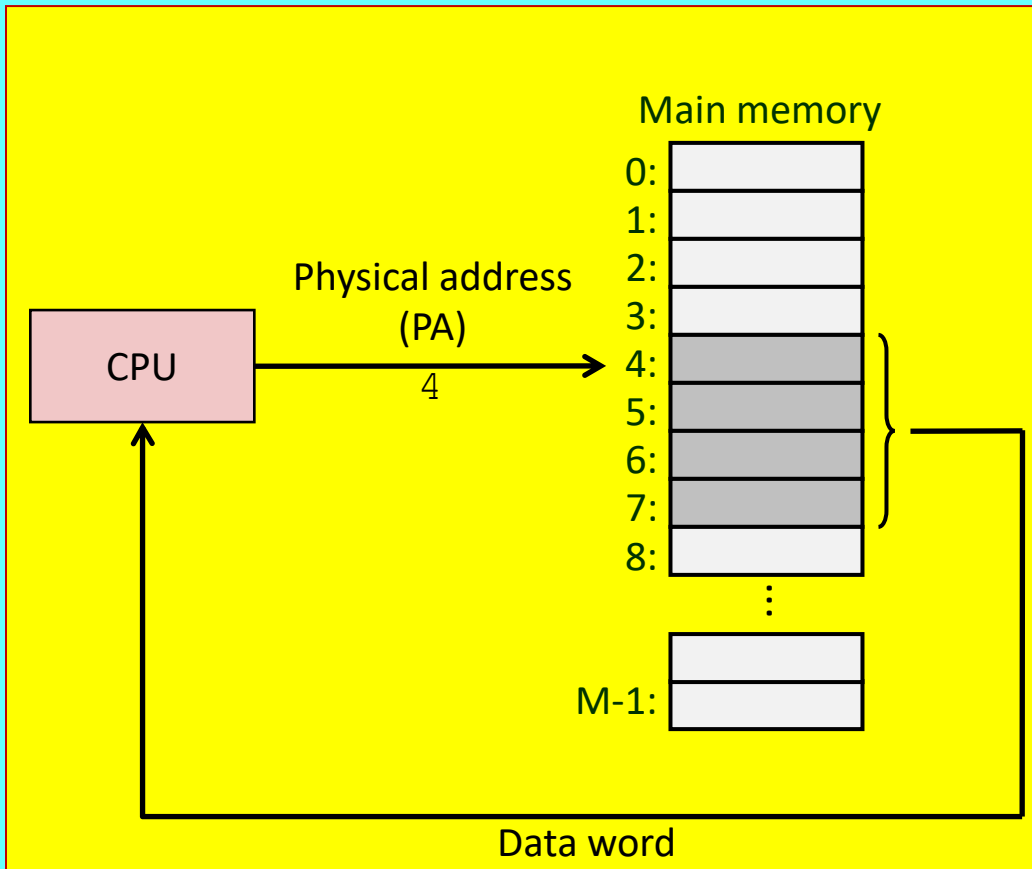
VIRTUAL MEMORY

- ★ In order to manage memory more efficiently and with fewer errors, modern systems provide an abstraction of main memory known as **virtual memory(VM)**.
- ★ **Virtual memory is an elegant interaction of hardware exceptions, hardware address translation, main memory, disk files, and kernel software that provides each process with a large, uniform, and private address space.**
- ★ With one clean mechanism, virtual memory provides three important capabilities.
 - (1) **It uses main memory efficiently by treating it as a cache for an address space stored on disk, keeping only the active areas in main memory, and transferring data back and forth between disk and memory as needed.**
 - (2) **It simplifies memory management by providing each process with an uniform address space.**
 - (3) **It protects the address space of each process from corruption by other processes.**

Why Virtual Memory (VM)?

- ★ **Uses main memory efficiently**
 - Use DRAM as a cache for parts of a virtual address space
- ★ **Simplifies memory management**
 - Each process gets the same uniform linear address space
- ★ **Isolates address spaces**
 - One process can't interfere with another's memory
 - User program cannot access privileged kernel information and code

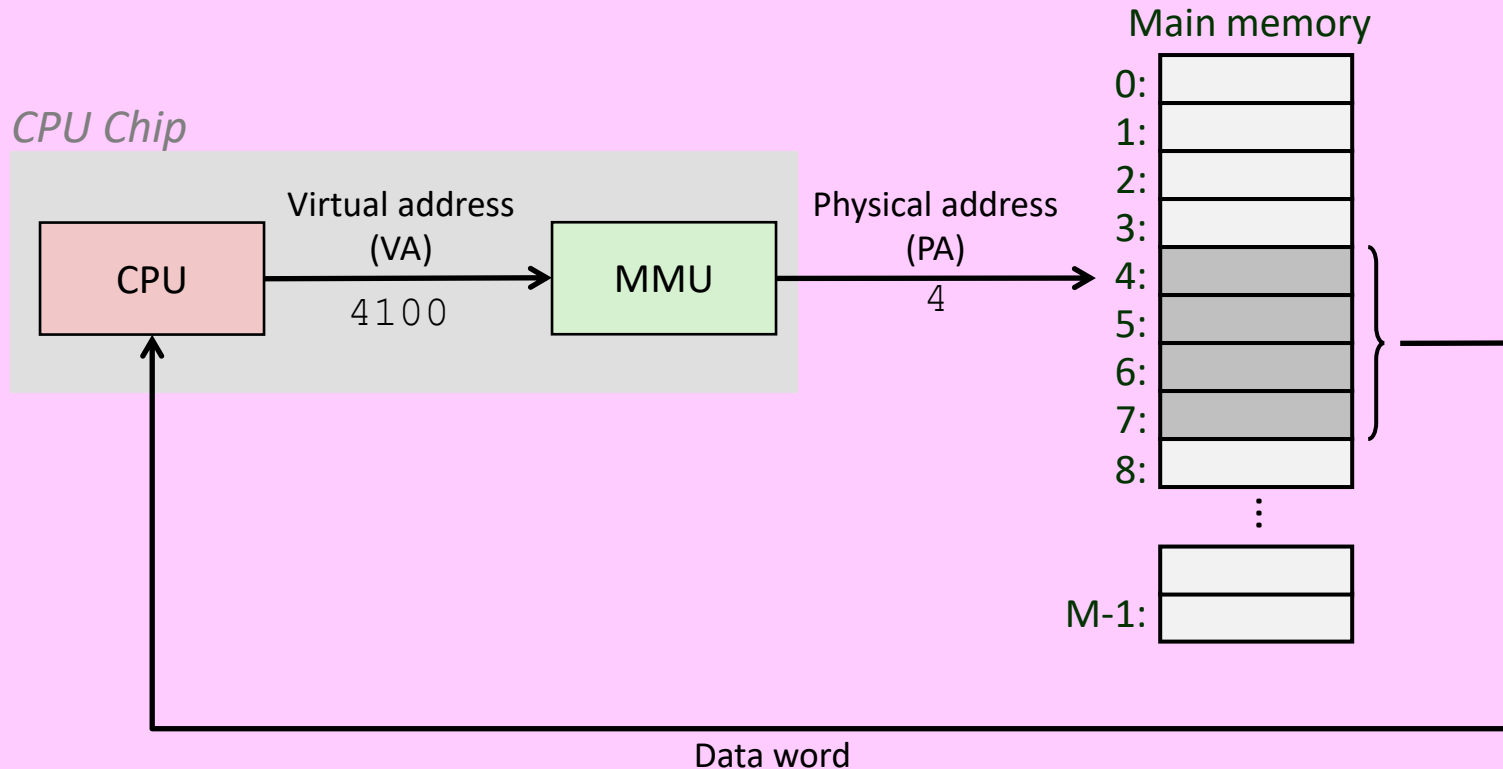
A System Using Physical Addressing



- The main memory of a computer system is organized as an array of **M contiguous** byte-sized cells.
- Each byte has a unique physical address (PA).
- The first byte has an address of 0, the next byte an address of 1, the next byte an address of 2, and so on.
- Given this simple organization, the most natural way for a CPU to access memory would be to use physical addresses.
- We call this approach physical addressing.
- Figure shows an example of physical addressing in the context of a load instruction that reads the word starting at physical address 4.
- When the CPU executes the load instruction, it generates an effective physical address and passes it to main memory over the memory bus.
- The main memory fetches the 4-byte word starting at physical address 4 and returns it to the CPU, which stores it in a register.

★ Used in “simple” systems like embedded microcontrollers in devices like cars, elevators, and digital **picture** frames

A System Using Virtual Addressing



▪Used in all modern servers, laptops, and smart phones

•One of the great ideas in computer science

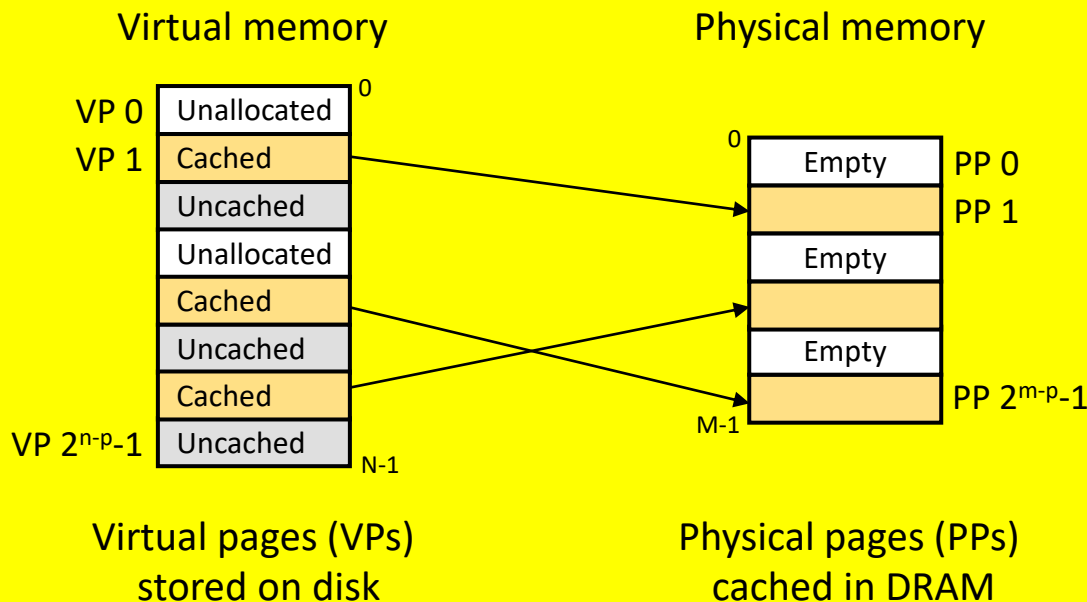
- With virtual addressing, the CPU accesses main memory by generating a virtual address (VA), which is converted to the appropriate physical address before being sent to the memory.
- The task of converting a virtual address to a physical one is known as **address translation**.
- Like exception handling, address translation requires close cooperation between the CPU hardware and the operating system.
- **Dedicated hardware on the CPU chip called the memory management unit (MMU) translates virtual addresses on the fly, using a look-up table stored in main memory whose contents are managed by the operating system.**

Address Spaces

- ★ **Virtual address space:** Set of $N = 2^n$ virtual addresses. In a system with virtual memory, the CPU generates virtual addresses from an address space of $N = 2^n$ addresses called the **virtual address space (also termed as linear address space)**
 $\{0, 1, 2, 3, \dots, N-1\}$
- ★ Modern systems typically support either 32-bit or 64-bit virtual address spaces.
- ★ **Physical address space:** Set of $M = 2^m$ physical addresses
 $\{0, 1, 2, 3, \dots, M-1\}$
- ★ Each byte of main memory has a virtual address chosen from the virtual address space, and a physical address chosen from the physical address space.

VM as a Tool for Caching

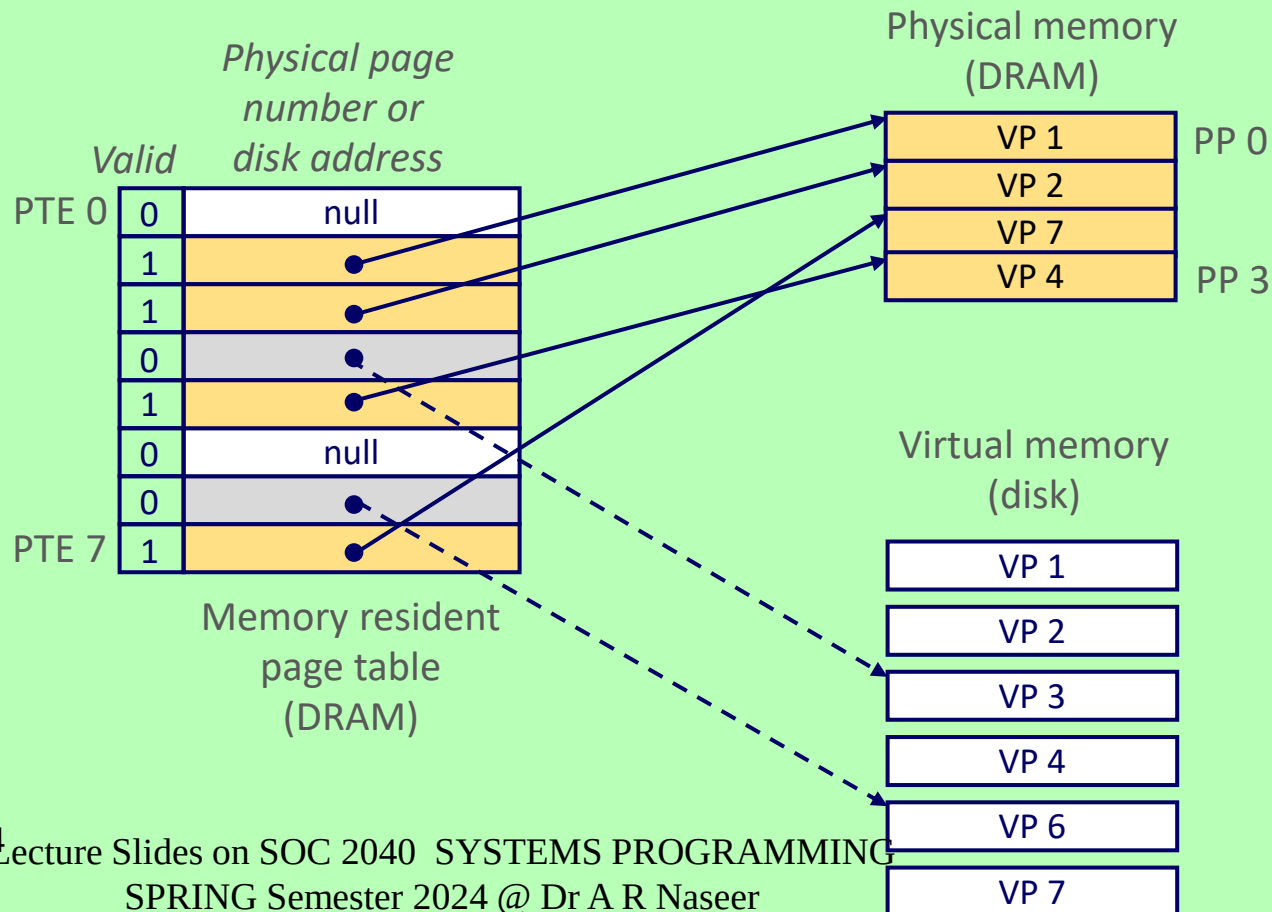
- ★ Conceptually, *virtual memory* is an array of N contiguous bytes stored on disk.
- ★ The contents of the array on disk are cached in *physical memory* (*DRAM cache*)
 - These cache blocks are called *pages* (size is $P = 2^p$ bytes)



- The example in Figure shows a small virtual memory with eight virtual pages.
- Virtual pages 0 and 3 have not been allocated yet, and thus do not yet exist on disk.
- Virtual pages 1, 4 and 6 are cached in physical memory.
- Pages 2, 5, and 7 are allocated, but are not currently cached in main memory.

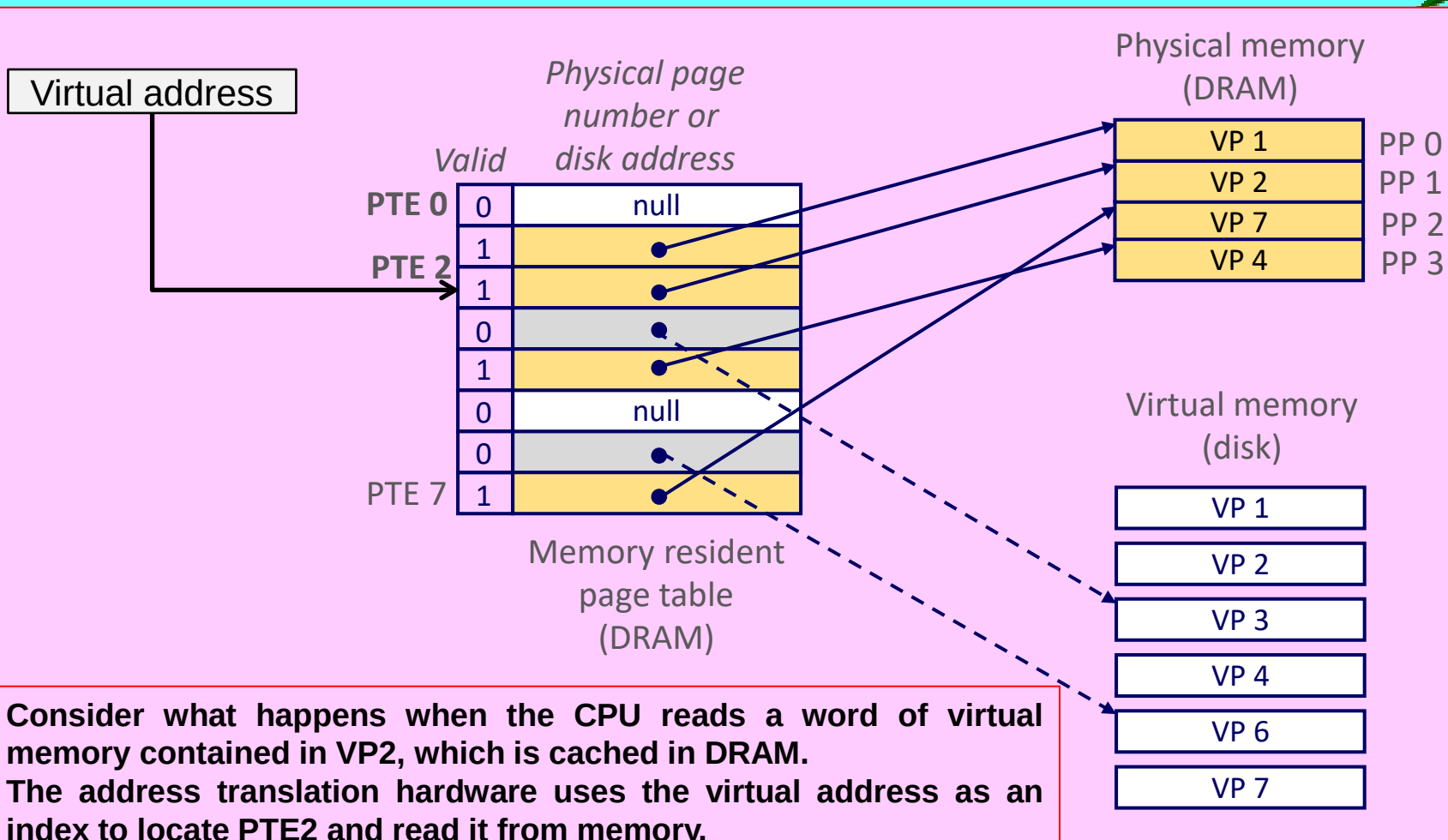
Enabling Data Structure: Page Table

- ★ A *page table* is an array of page table entries (PTEs) that maps virtual pages to physical pages.
 - Per-process kernel data structure in DRAM



Page Hit

* **Page hit:** reference to VM word that is in physical memory (DRAM cache hit)



Consider what happens when the CPU reads a word of virtual memory contained in VP2, which is cached in DRAM. The address translation hardware uses the virtual address as an index to locate PTE2 and read it from memory. Since the valid bit is set, the address translation hardware knows that VP2 is cached in memory. So it uses the physical memory address in the PTE(which points to the start of the cached page in PP1) to construct the physical address of the word.

Page Fault

- * **Page fault:** reference to VM word that is not in physical memory (DRAM cache miss)

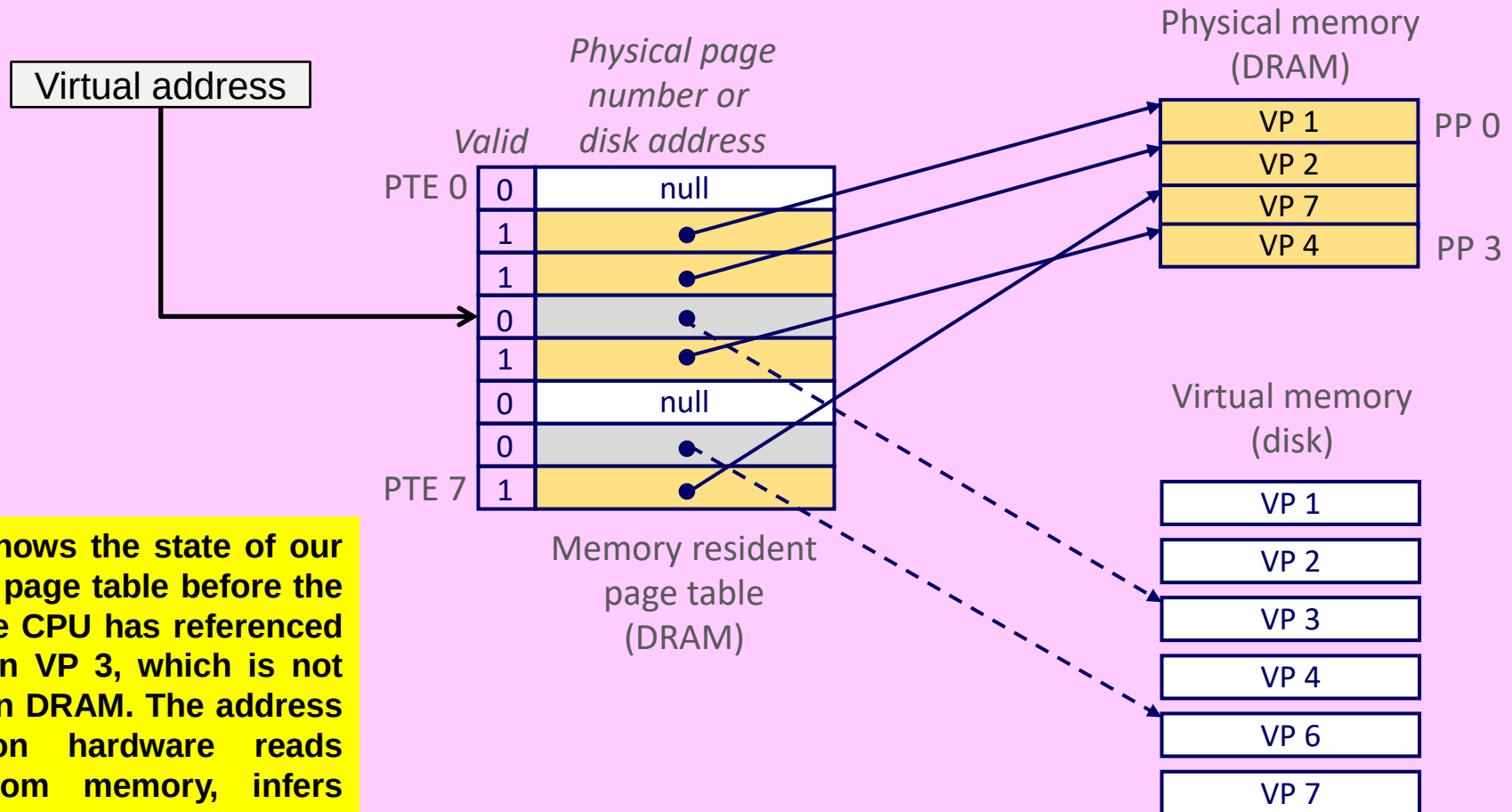
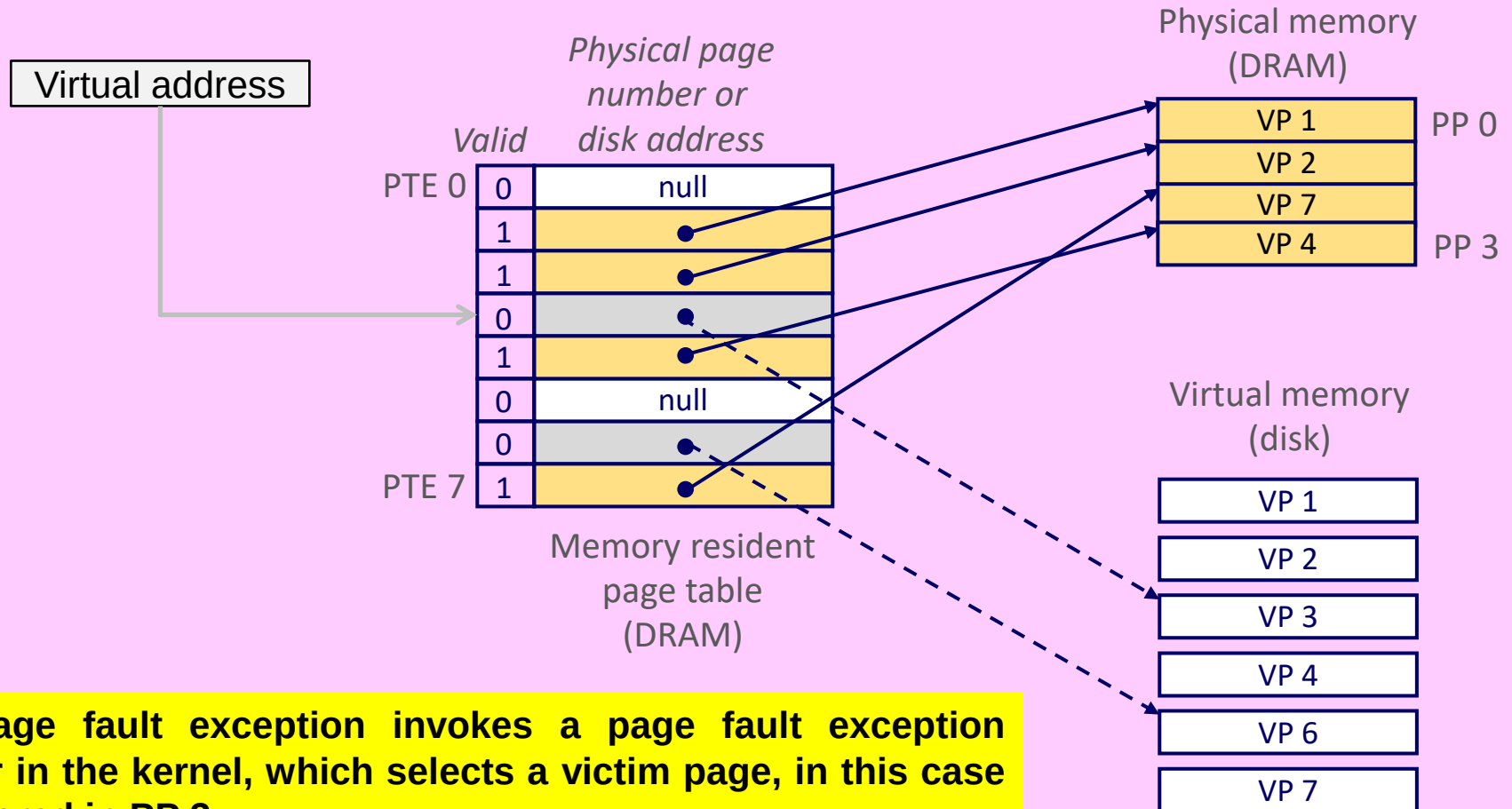


Figure shows the state of our example page table before the fault. The CPU has referenced a word in VP 3, which is not cached in DRAM. The address translation hardware reads PTE3 from memory, infers from the valid bit that VP3 is not cached, and triggers a page fault exception.

Handling Page Fault

- ★ Page miss causes page fault (an exception)
- ★ Page fault handler selects a victim to be evicted (here VP 4)



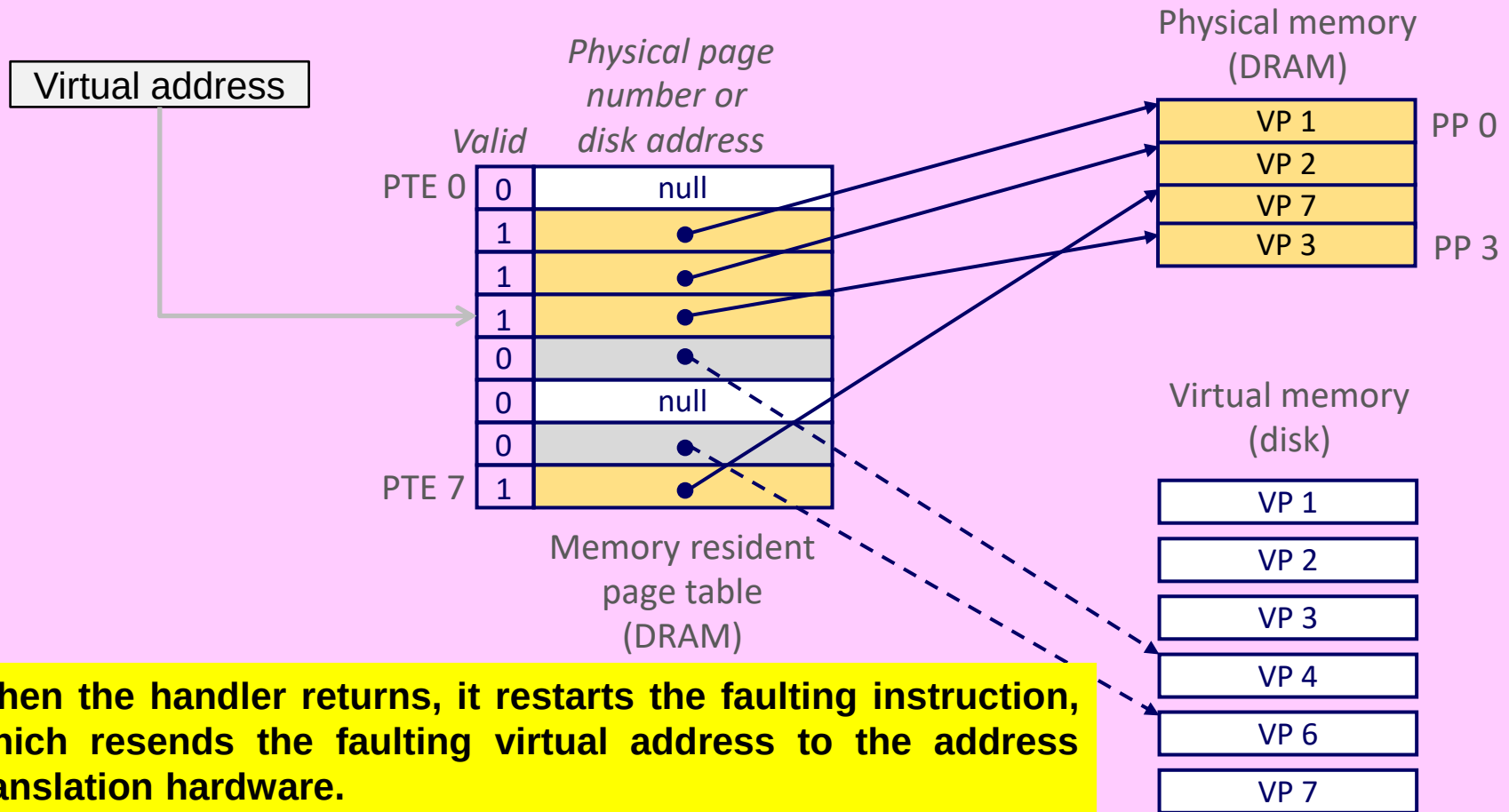
The page fault exception invokes a page fault exception handler in the kernel, which selects a victim page, in this case VP 4 stored in PP 3.

If VP 4 has been modified, then the kernel copies it back to disk. In either case, the kernel modifies the page table entry for VP 4 to reflect the fact that VP 4 is no longer cached in main memory.

Next, the kernel copies VP 3 from disk to PP 3 in memory, updates PTE 3, and then returns.

Handling Page Fault

- * Page miss causes page fault (an exception)
- * Page fault handler selects a victim to be evicted (here VP 4)



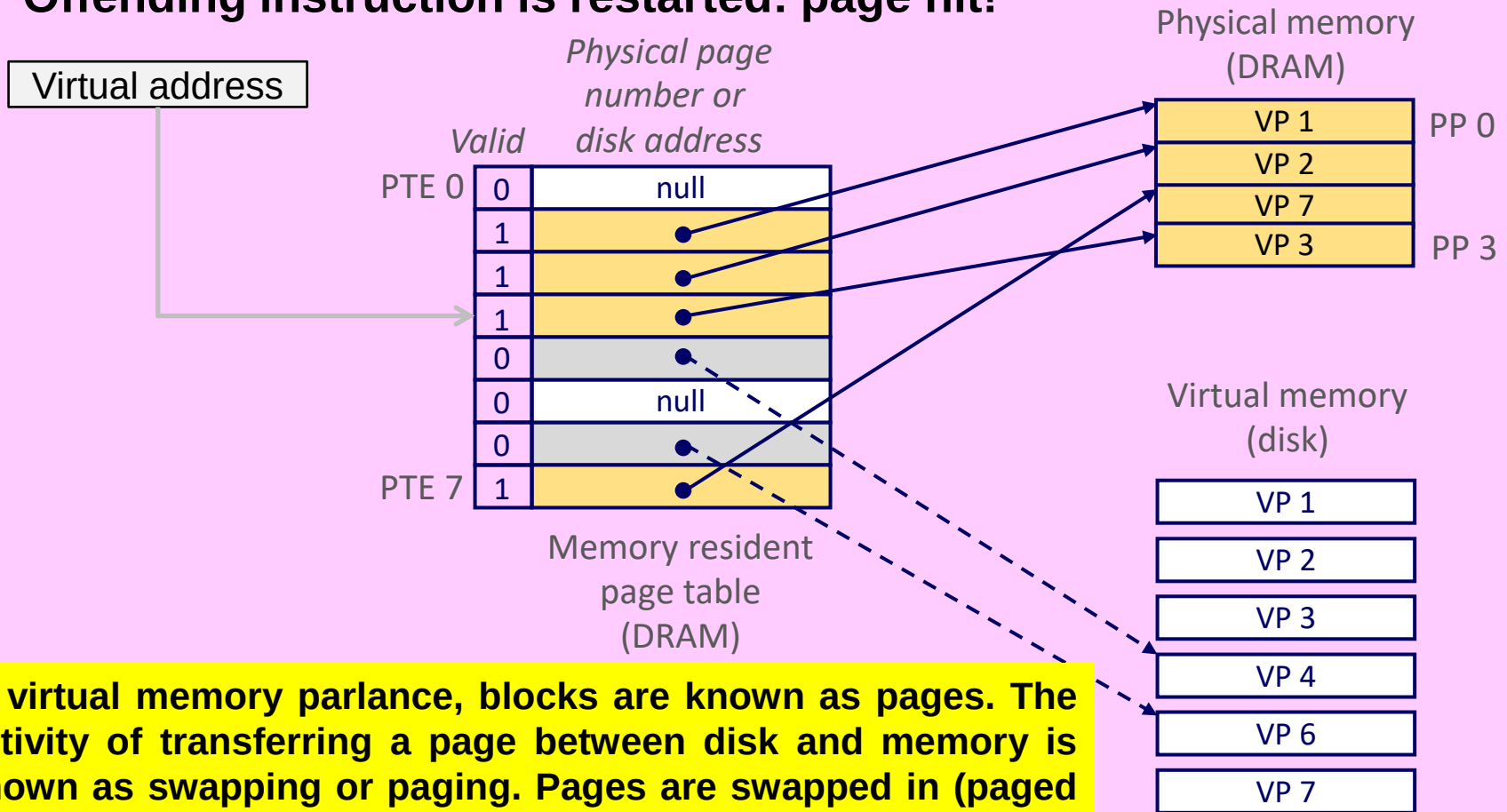
When the handler returns, it restarts the faulting instruction, which resends the faulting virtual address to the address translation hardware.

But now, VP3 is cached in main memory, and the page hit is handled normally by the address translation hardware.

Figure shows the state of our example page table after the page fault.

Handling Page Fault

- * Page miss causes page fault (an exception)
- * Page fault handler selects a victim to be evicted (here VP 4)
- * Offending instruction is restarted: page hit!



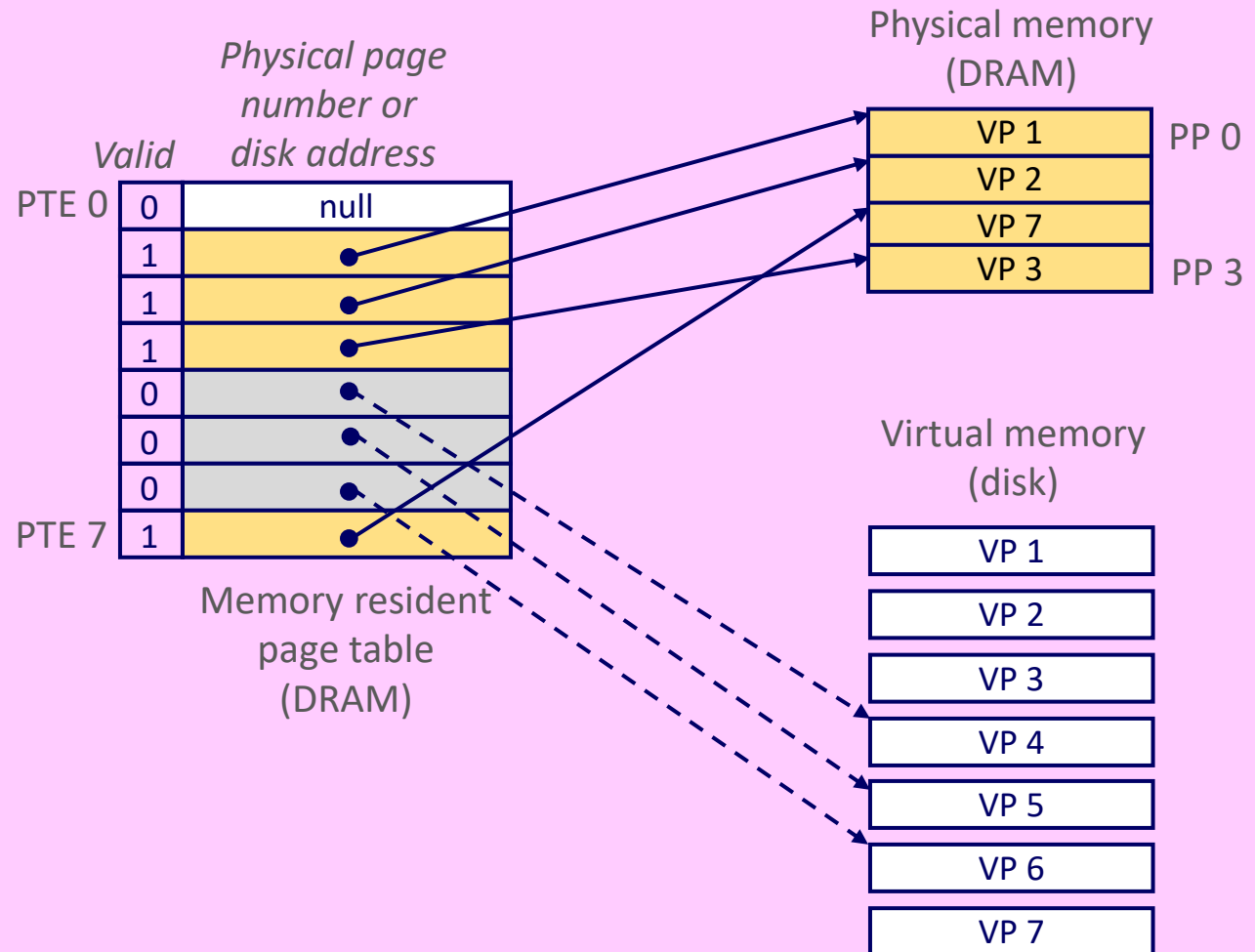
In virtual memory parlance, blocks are known as pages. The activity of transferring a page between disk and memory is known as swapping or paging. Pages are swapped in (paged in) from disk to DRAM, and swapped out (paged out) from DRAM to disk. **The strategy of waiting until the last moment to swap in a page, when a miss occurs, is known as demand paging.**

Allocating Pages

- ★ Allocating a new page (VP 5) of virtual memory.

Figure shows the effect on our example page table when the operating system allocates a new page of virtual memory, for example, as a result of calling malloc.

In the example, VP 5 is allocated by creating room on disk and updating PTE 5 to point to the newly created page on disk

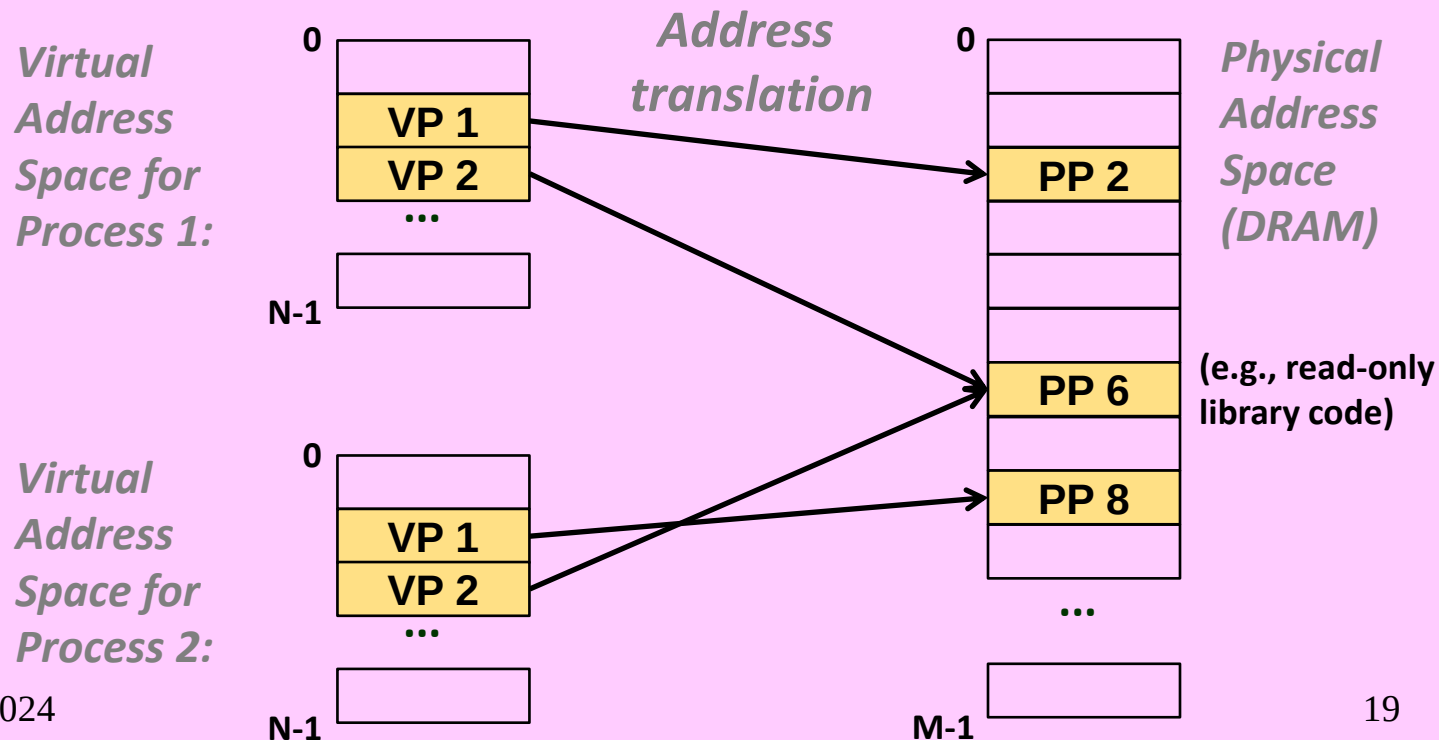


PRINCIPLE OF LOCALITY

- ★ Although the total number of distinct pages that programs reference during an entire run might exceed the total size of physical memory, the principle of locality promises that at any point in time they will tend to work on a smaller set of active pages known as the working set or resident set.
- ★ After an initial overhead where the working set is paged into memory, subsequent references to the working set result in hits, with no additional disk traffic.
- ★ As long as our programs have good temporal locality, virtual memory systems work quite well. But of course, not all programs exhibit good temporal locality.
- **Thrashing :**
 - ★ If the working set size exceeds the size of physical memory, then the program can produce an unfortunate situation known as thrashing, where pages are swapped in and out continuously.
 - ★ Although virtual memory is usually efficient, if a program's performance slows to a crawl, the wise programmer will consider the possibility that it is thrashing.

VM as a Tool for Memory Management

- ★ Key idea: each process has its own virtual address space
 - It can view memory as a simple linear array
 - Mapping function scatters addresses through physical memory
 - ★ Well-chosen mappings can improve locality



Simplifying Linking and Loading



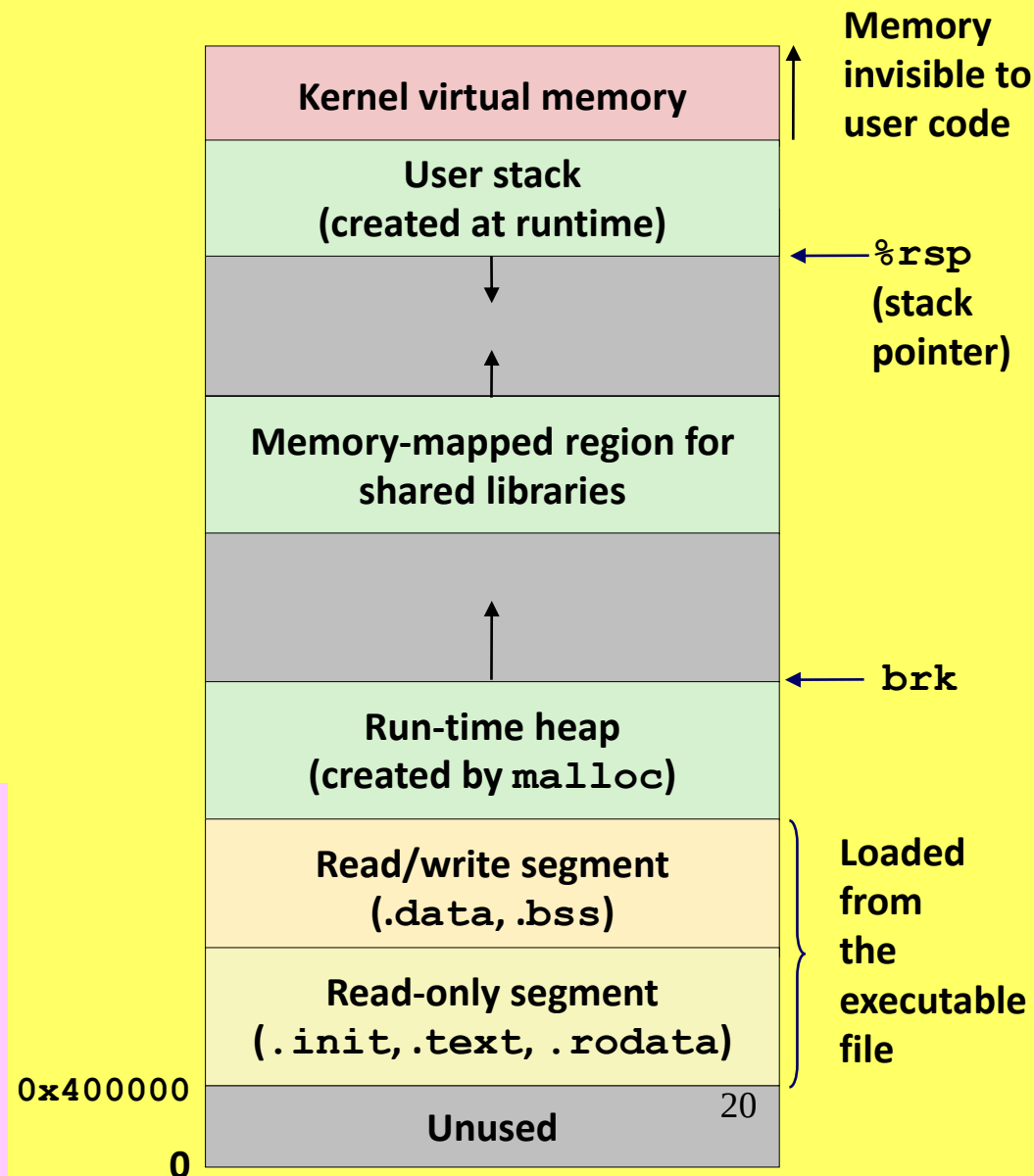
★ Linking

- Each program has similar virtual address space
- Code, data, and heap always start at the same addresses.

★ Loading

- **execve** allocates virtual pages for `.text` and `.data` sections & creates PTEs marked as invalid
- The `.text` and `.data` sections are copied, page by page, on demand by the virtual memory system

The notion of mapping a set of contiguous virtual pages to an arbitrary location in an arbitrary file is known as **memory mapping**. Unix provides a system call called **mmap** that allows application programs to do their own memory mapping.



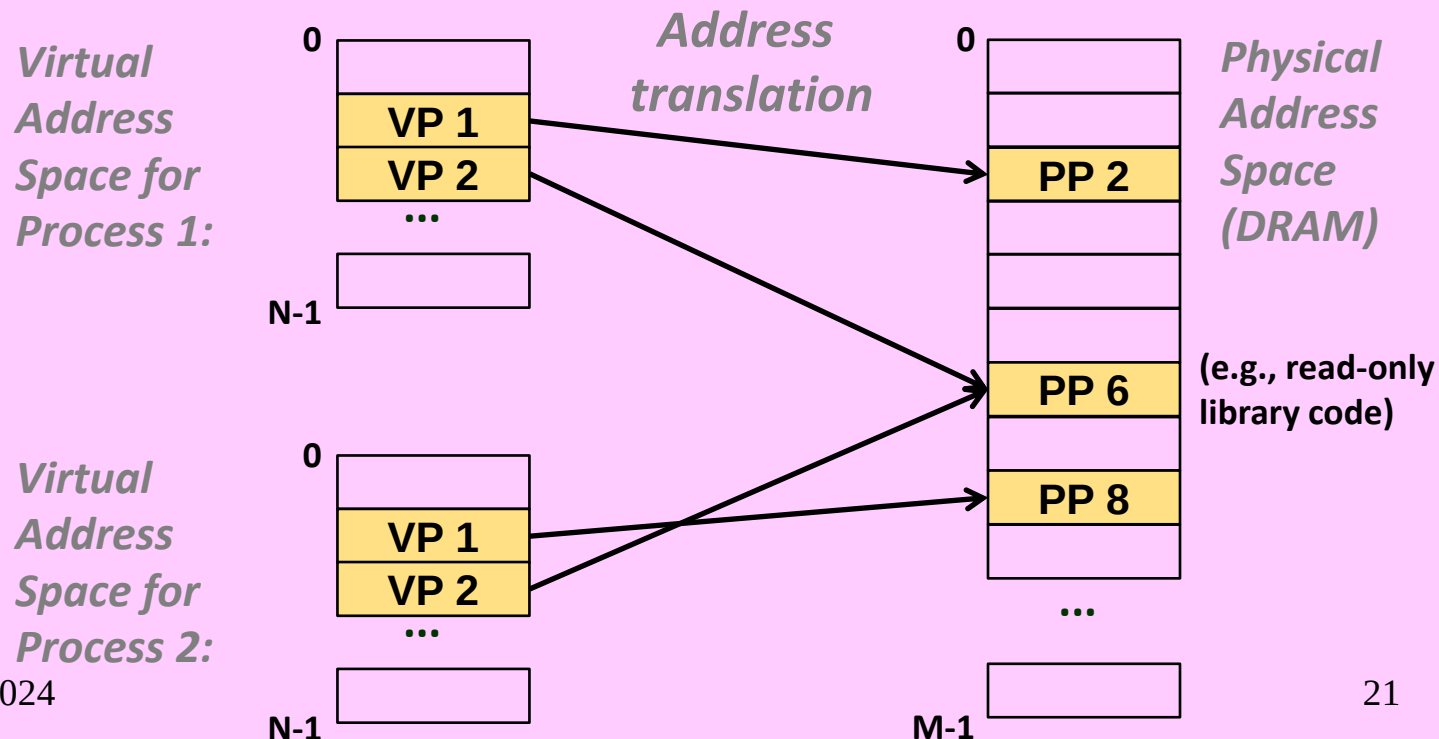
VM as a Tool for Memory Management

★ Simplifying memory allocation

- Each virtual page can be mapped to any physical page
- A virtual page can be stored in different physical pages at different times

★ Sharing code and data among processes

- Map virtual pages to the same physical page (here: PP 6)



Simplifying Sharing & Memory Allocation

Simplifying sharing

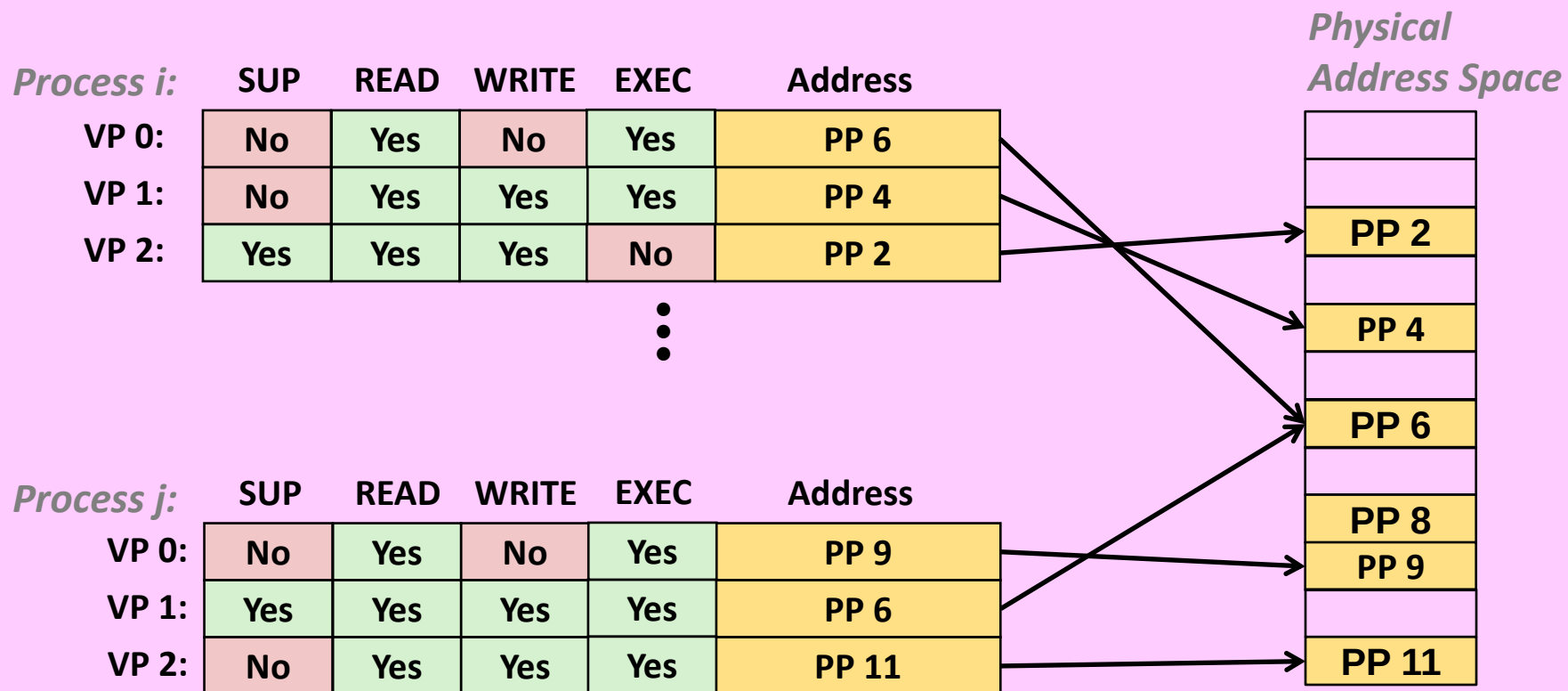
- ★ Separate address spaces provide the operating system with a consistent mechanism for managing sharing between user processes and the operating system itself.
- ★ In general, each process has its own private code, data, heap, and stack areas that are not shared with any other process.
- ★ In this case, the operating system creates page tables that map the corresponding virtual pages to disjoint physical pages..

Simplifying memory allocation

- ★ Virtual memory provides a simple mechanism for allocating additional memory to user processes.
- ★ When a program running in a user process requests additional heap space (e.g., as a result of calling malloc), the operating system allocates an appropriate number, say, k , of contiguous virtual memory pages, and maps them to k arbitrary physical pages located anywhere in physical memory.
- ★ Because of the way page tables work, there is no need for the operating system to locate k contiguous pages of physical memory.
- ★ The pages can be scattered randomly in physical memory.

VM as a Tool for Memory Protection

- ★ Extend PTEs with permission bits
- ★ MMU checks these bits on each access



SOC 2040

SYSTEM PROGRAMMING

VIRTUAL MEMORY

ADDRESS TRANSLATION

VM Address Translation

- ★ **Virtual Address Space**

- $VAS = \{0, 1, \dots, N-1\}$

- ★ **Physical Address Space**

- $PAS = \{0, 1, \dots, M-1\}$

- ★ **Address Translation**

- ★ Formally, address translation is a mapping between the elements of an N element virtual address space (VAS) and an M -element physical address space (PAS),

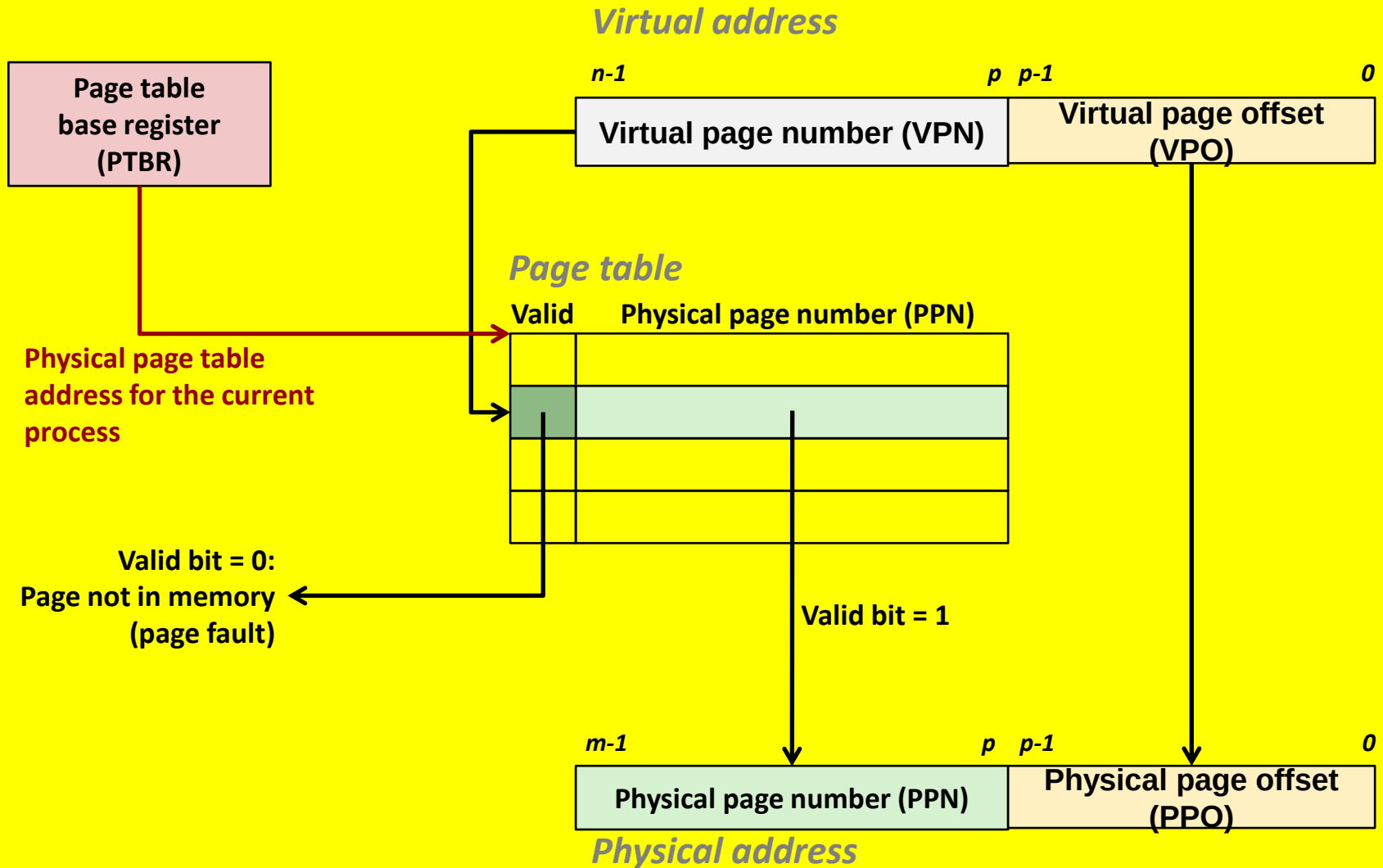
- **$MAP: VAS \rightarrow PAS \cup \{\emptyset\}$**

- For virtual address A :

- ★ **$MAP(A) = A'$** if data at virtual address A is at physical address A' in PAS

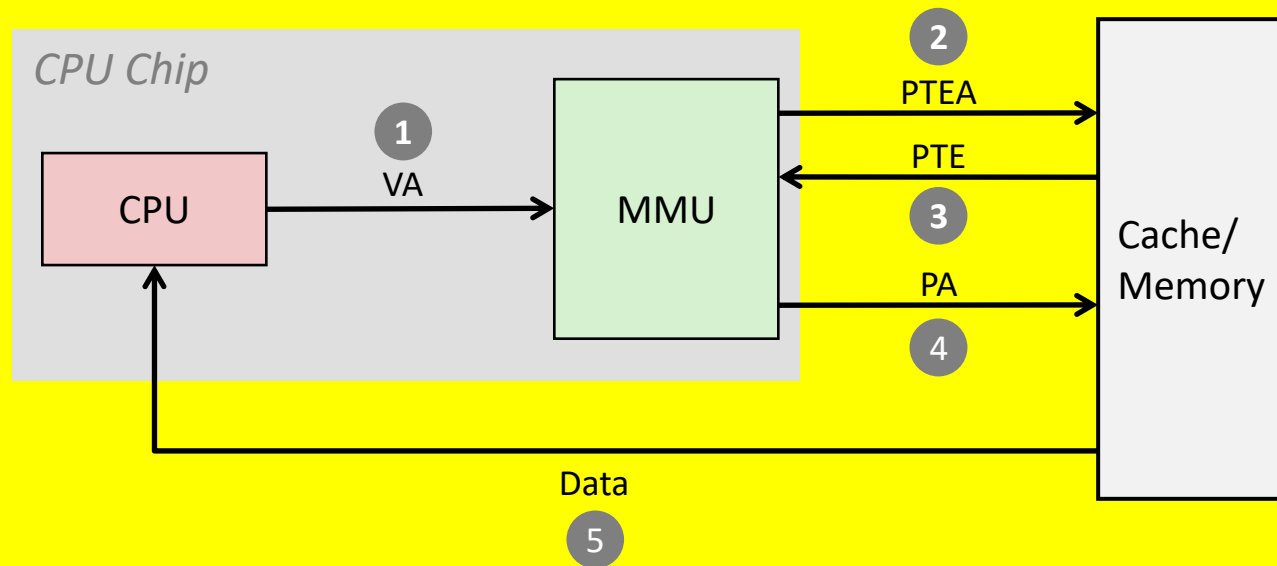
- ★ **$MAP(A) = \emptyset$** if data at virtual address A is not in physical memory

Address Translation With a Page Table



Address Translation: Page Hit

Figure shows the steps that the CPU hardware performs when there is a page hit.



1) Processor sends virtual address to MMU

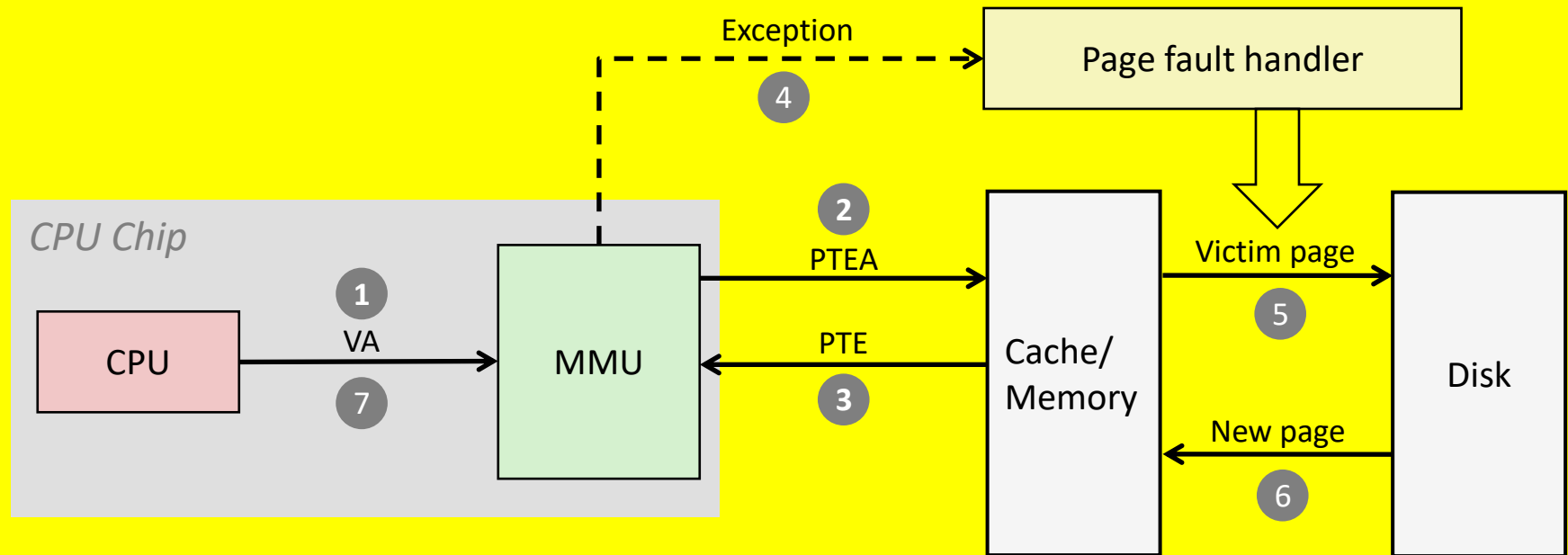
2-3) MMU fetches PTE from page table in memory

4) MMU constructs the physical address & sends physical address to cache/memory

5) Cache/memory returns the requested data word to processor

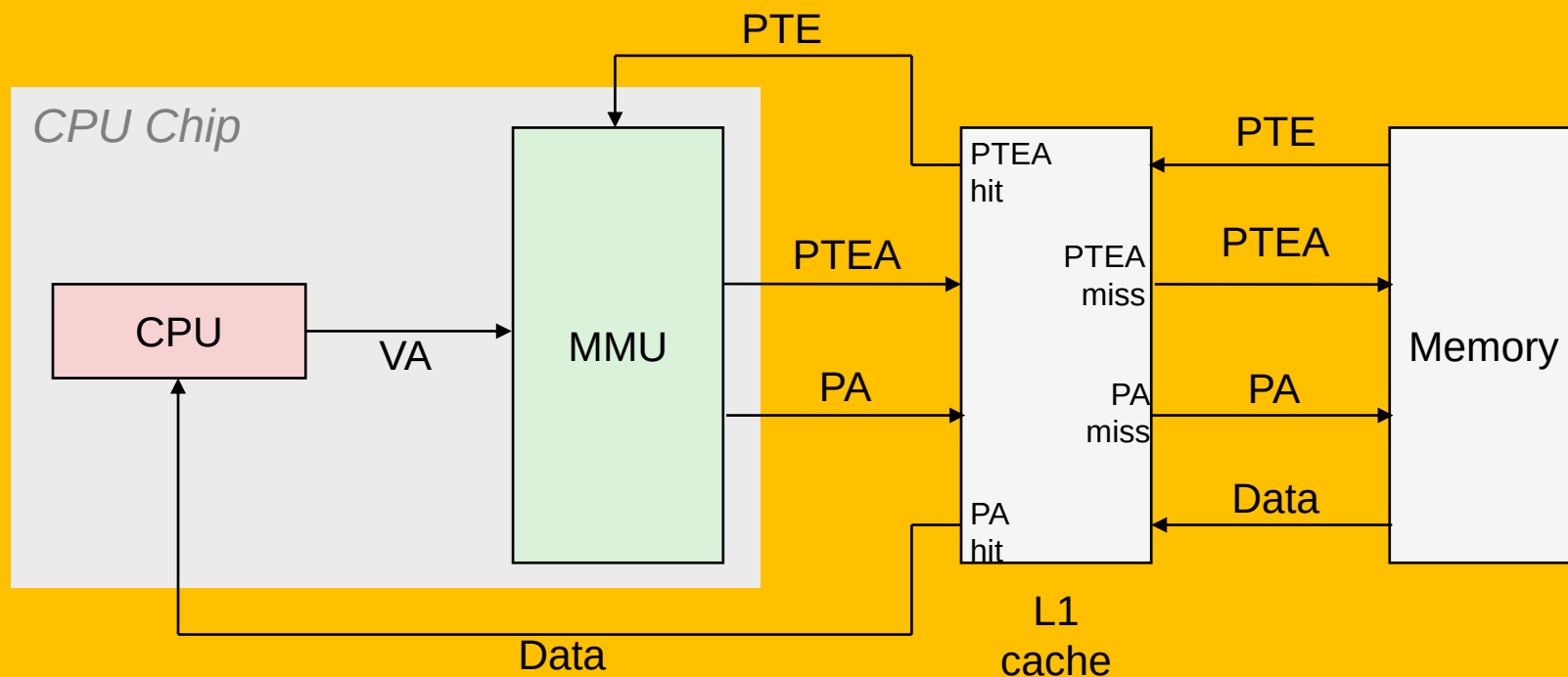
Address Translation: Page Fault

Unlike a page hit, which is handled entirely by hardware, handling a page fault requires cooperation between hardware and the operating system kernel



- 1) Processor sends virtual address to MMU
- 2-3) MMU fetches PTE from page table in memory
- 4) Valid bit is zero, so MMU triggers page fault exception
- 5) Handler identifies victim (and, if dirty, pages it out to disk)
- 6) Handler pages in new page and updates PTE in memory
- 7) Handler returns to original process, restarting faulting instruction

Integrating VM and Cache



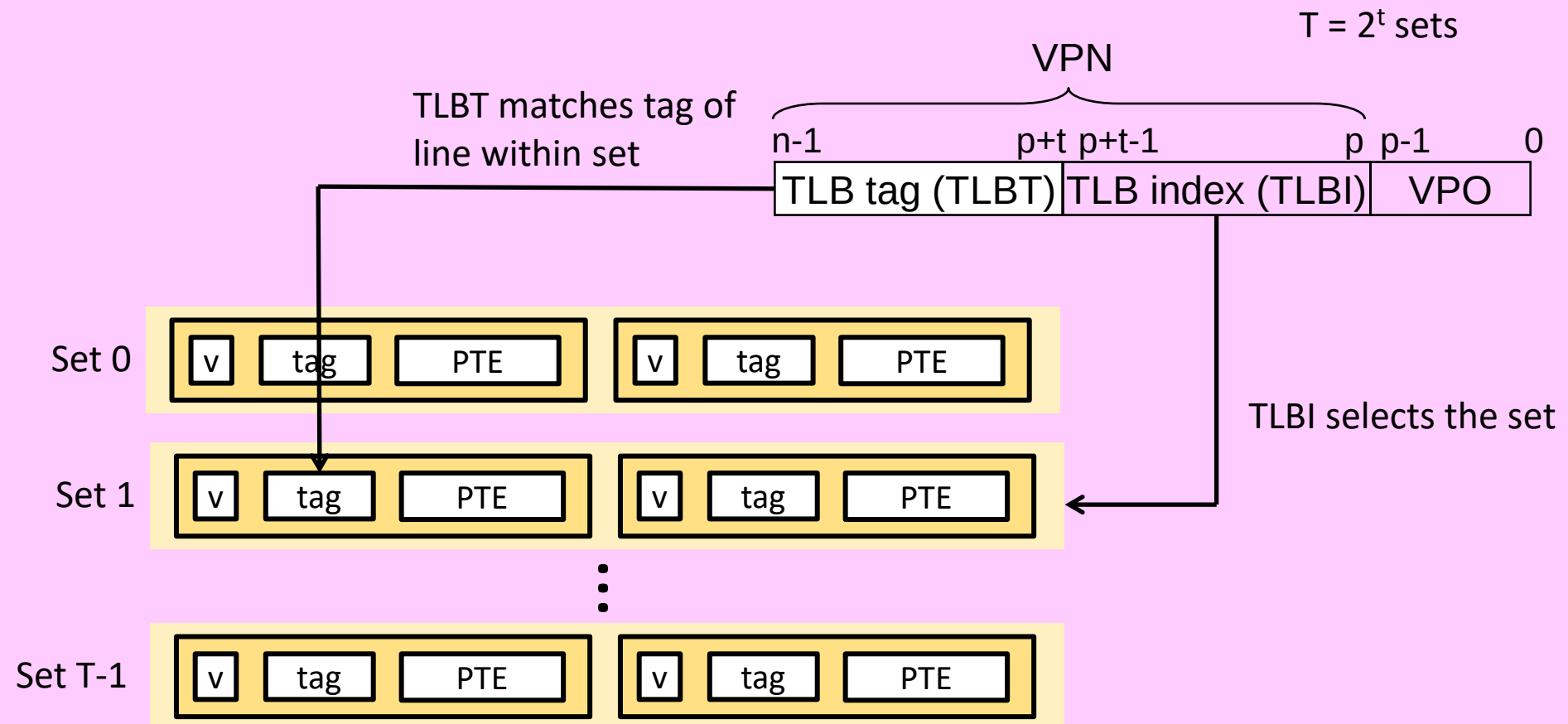
VA: *virtual* address, PA: physical address, PTE: page table entry, PTEA = PTE address

Speeding up Translation with a TLB

- ★ **Page table entries (PTEs) are cached in L1 like any other memory word**
 - PTEs may be evicted by other data references
 - PTE hit still requires a small L1 delay
- ★ **Solution: *Translation Lookaside Buffer (TLB)***
 - Small set-associative hardware cache in MMU
 - Maps virtual page numbers to physical page numbers
 - Contains complete page table entries for small number of pages

Accessing the TLB

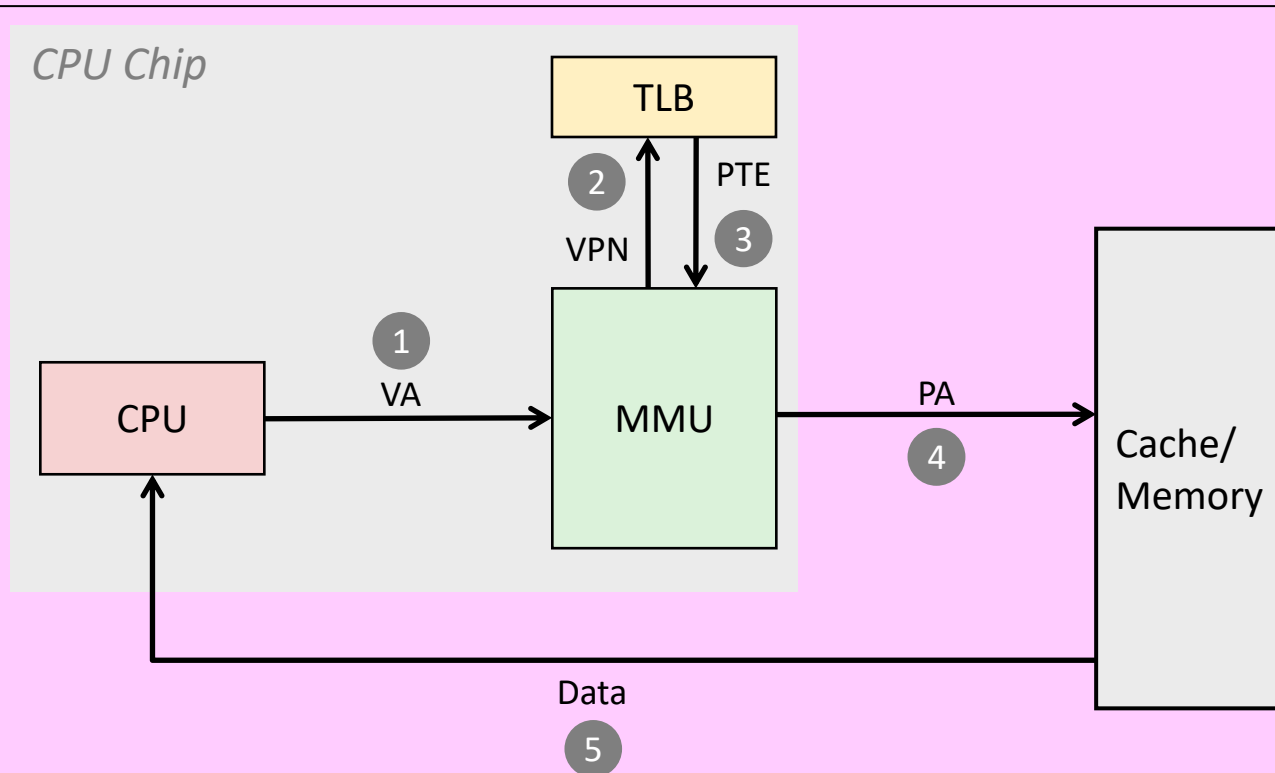
- ★ MMU uses the VPN portion of the virtual address to access the TLB:



TLB Hit

Figure shows the steps involved when there is a TLB hit (the usual case). The key point here is that all of the address translation steps are performed inside the on-chip MMU, and thus are fast.

A TLB hit eliminates a memory access



Step 1: The CPU generates a virtual address.

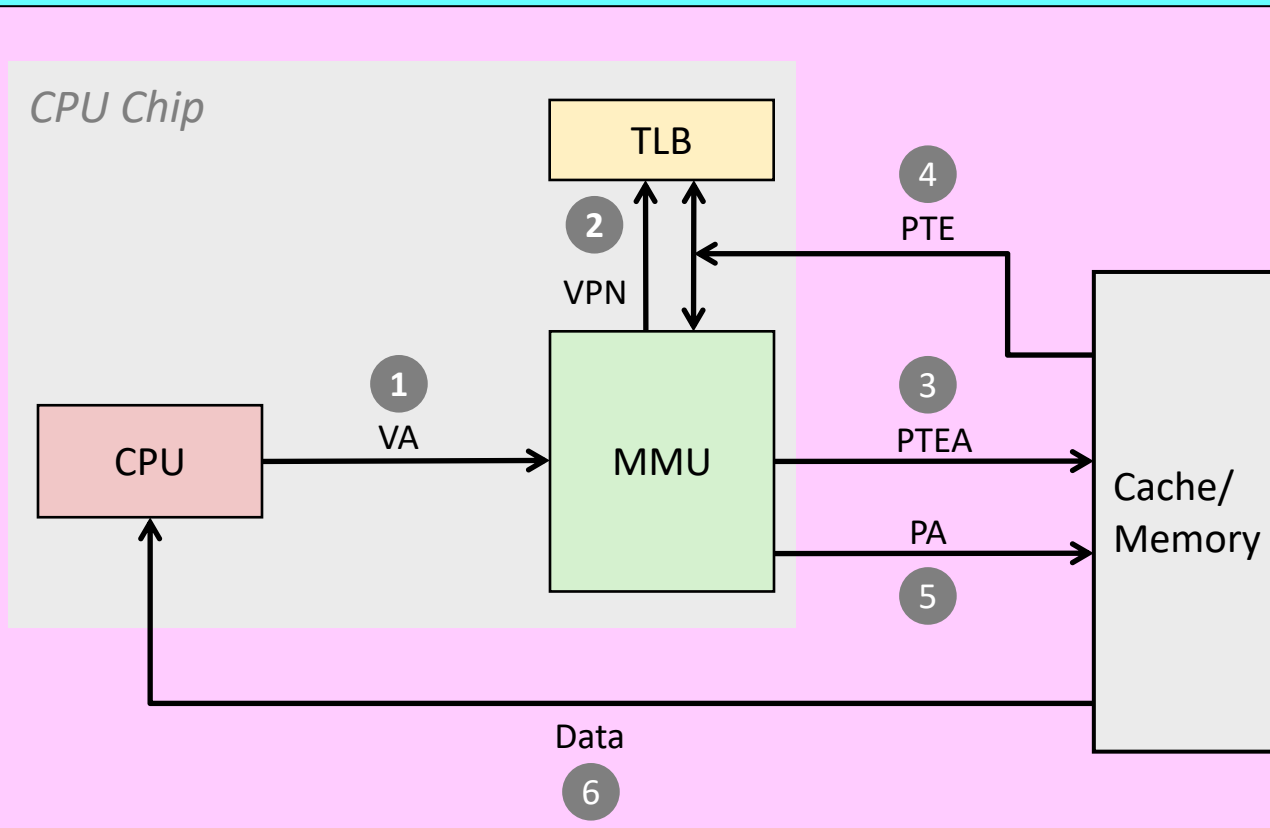
Steps 2 and 3: The MMU fetches the appropriate PTE from the TLB.

Step 4: The MMU translates the virtual address to a physical address and sends it to the cache/main memory.

Step 5: The cache/main memory returns the requested data word to the CPU.

TLB Miss

- When there is a TLB miss, then the MMU must fetch the PTE from the L1cache or Page Table in Main memory, as shown in Figure
- The newly fetched PTE is stored in the TLB, possibly overwriting an existing entry.

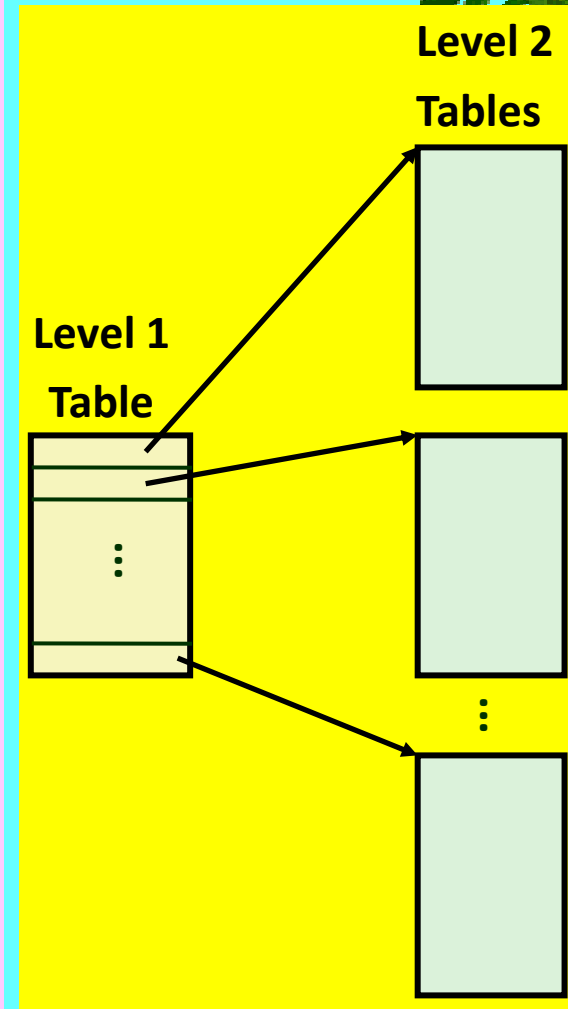


A TLB miss incurs an additional memory access (the PTE)

Fortunately, TLB misses are rare. Why?

Multi-Level Page Tables

- ★ Suppose:
 - 4KB (2^{12}) page size, 48-bit address space, 8-byte PTE
- ★ Problem:
 - Would need a 512 GB page table!
 - ★ $2^{48} * 2^{-12} * 2^3 = 2^{39}$ bytes
- ★ Common solution: Multi-level page table
- ★ Example: 2-level page table
 - Level 1 table: each PTE points to a page table (always memory resident)
 - Level 2 table: each PTE points to a page



Multi-Level Page Tables

The common approach for compacting the page table is to use a hierarchy of page tables instead. The idea is easiest to understand with a concrete example.

Consider a 32-bit virtual address space partitioned into 4 KB pages, with page table entries that are 4bytes each.

Suppose also that at this point in time the virtual address space has the following form:

The first 2K pages of memory are allocated for code and data, the next 6K pages are unallocated, the next 1023 pages are also unallocated, and the next page is allocated for the user stack.

Figure shows how we might construct a two-level page table hierarchy for this virtual address space.

Each PTE in the level-1 table is responsible for mapping a 4MB chunk of the virtual address space, where each chunk consists of 1024 contiguous pages.

For example, PTE 0 maps the first chunk, PTE 1 the next chunk, and so on.

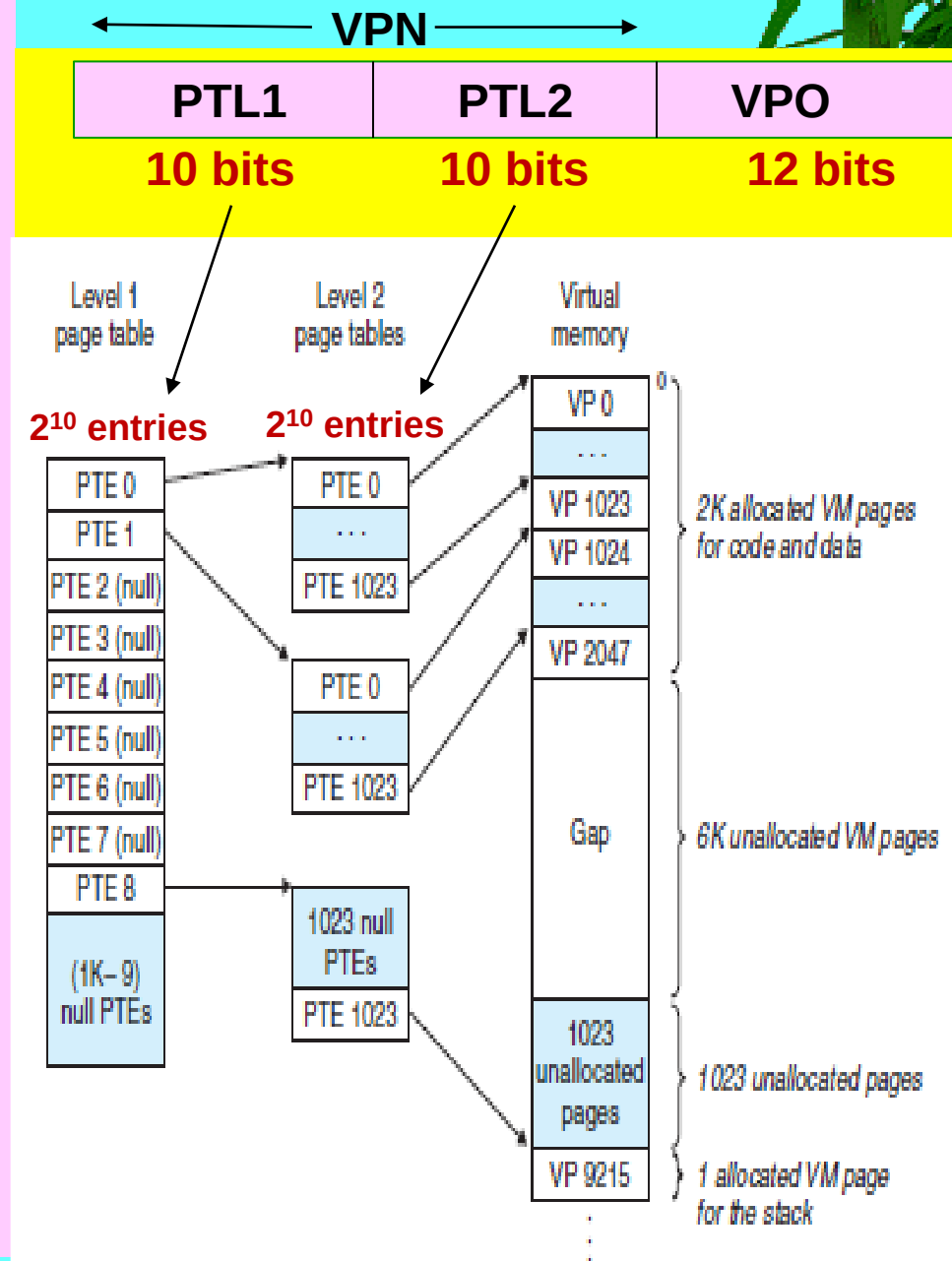
Given that the address space is 4 GB, 1024 PTEs are sufficient to cover the entire space.

If every page in chunk i is unallocated, then level1 PTE $_i$ is null.

For example, in Figure, chunks 2–7 are unallocated.

However, if atleast one page in chunk i is allocated, then level 1 PTE i points to the base of a level 2 page table.

For example, in Figure, all or portions of chunks 0,1,and 8 are allocated, so their level 1 PTEs point to level 2 page tables.



Multi-Level Page Tables

* Each PTE in a level 2 page table is responsible for mapping a 4 KB page of virtual memory, just as before when we looked at single-level page tables.

* Notice that with 4-byte PTEs, each level 1 and level 2 page table is 4K bytes, which conveniently is the same size as a page.

* This scheme reduces memory requirements in two ways

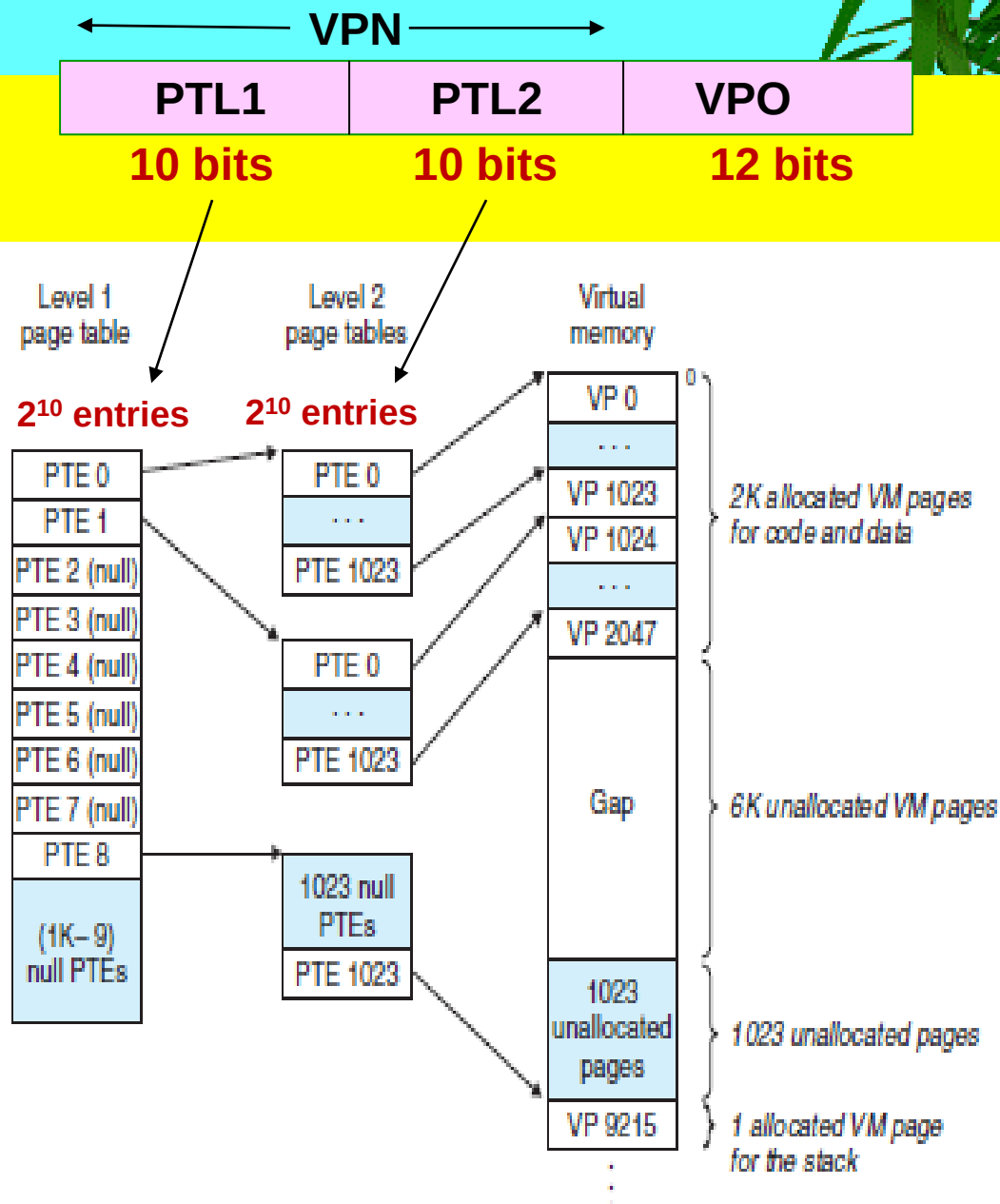
* First, if a PTE in the level1 table is null, then the corresponding level2 page table does not even have to exist.

* This represents a significant potential savings, since most of the 4GB virtual address space for a typical program is unallocated.

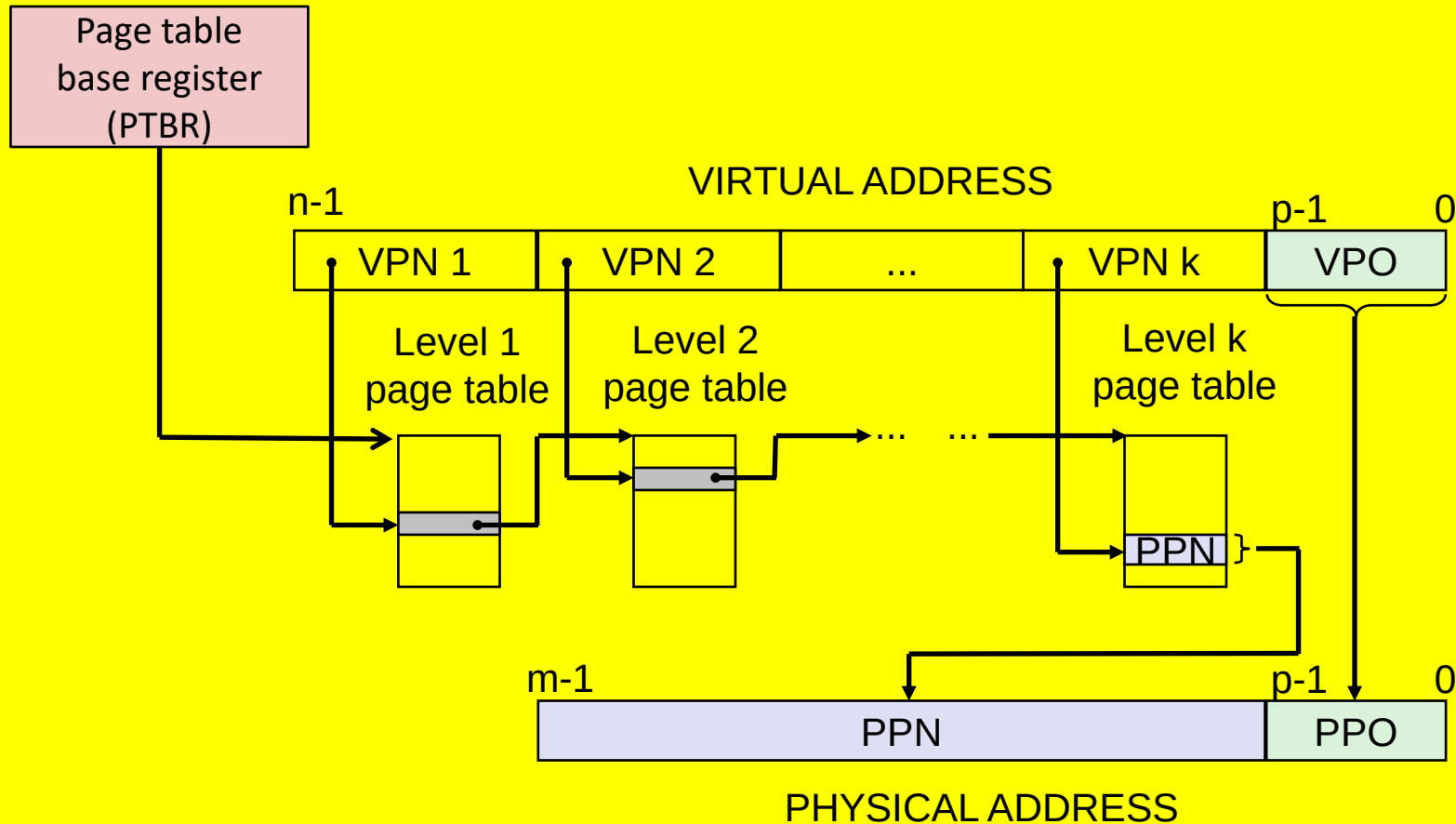
* Second, only the level 1 table needs to be in main memory at all times.

* The level2 page tables can be created and Paged in and out by the VM system as they are needed, which reduces pressure on main memory.

* Only the most heavily used level 2 page tables need to be cached in main memory.



Translating with a k-level Page Table



VIRTUAL MEMORY SYSTEMS

Simple memory system example

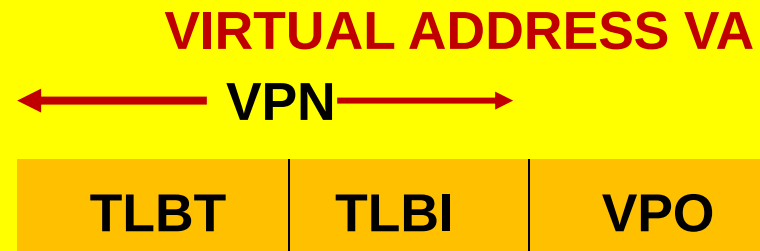
Review of Symbols

* Basic Parameters

- $N = 2^n$: Number of addresses in virtual address space
- $M = 2^m$: Number of addresses in physical address space
- $P = 2^p$: Page size (bytes)

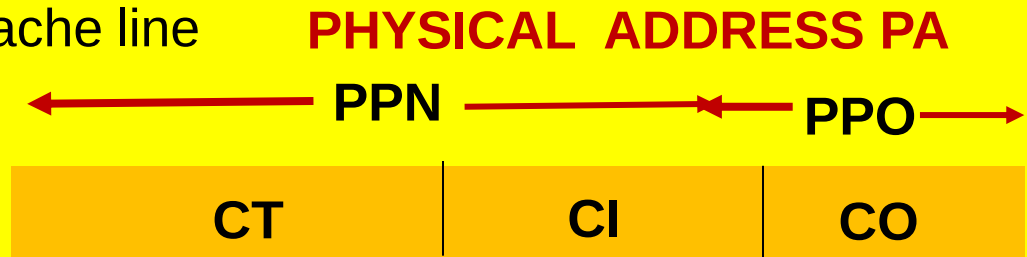
* Components of the virtual address (VA)

- **TLBI**: TLB index
- **TLBT**: TLB tag
- **VPO**: Virtual page offset
- **VPN**: Virtual page number



* Components of the physical address (PA)

- **PPO**: Physical page offset (same as VPO)
- **PPN**: Physical page number
- **CO**: Byte offset within cache line
- **CI**: Cache index
- **CT**: Cache tag



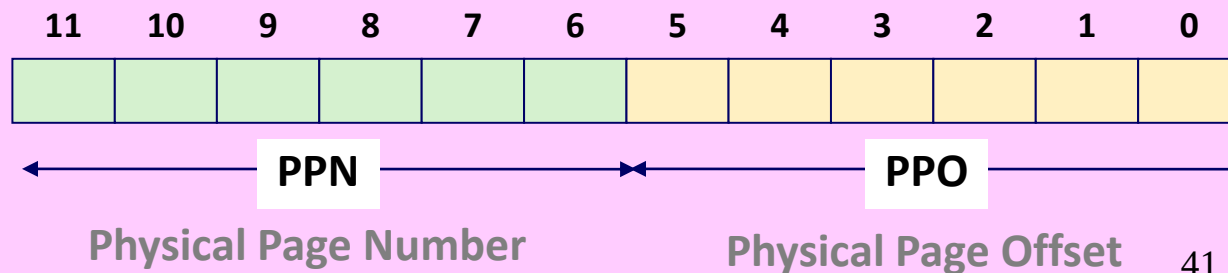
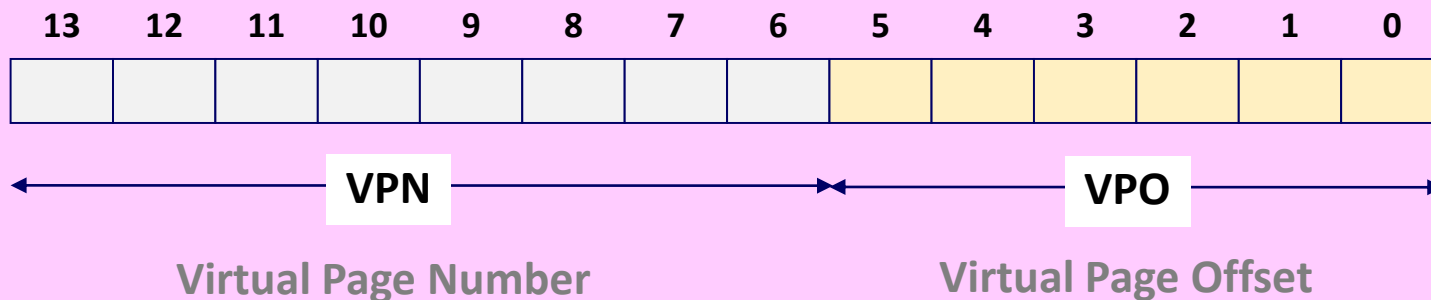
End-to-end Address Translation

- ★ Consider a concrete example of end-to-end address translation on a small system with a TLB and L1 d-cache.
- ★ To keep things manageable, we make the following assumptions:
 - . **The memory is byte addressable.**
 - . **Memory accesses are to 1-byte words (not 4-byte words).**
 - . **Virtual addresses are 14 bits wide ($n=14$).**
 - . **Physical addresses are 12 bits wide ($m=12$).**
 - . **The page size is 64 bytes ($P=64$).**
 - . **The TLB is four-way set associative with 16 total entries.**
 - . **The L1 d-cache is physically addressed and direct mapped, with a 4-byte line size and 16 total sets.**
- ★ Figure shows the formats of the virtual and physical addresses.
- ★ Since each page is $2^6=64$ bytes, the low-order 6 bits of the virtual and physical addresses serve as the VPO and PPO respectively.
- ★ The high-order 8 bits of the virtual address serve as the VPN.
- ★ The high-order 6 bits of the physical address serve as the PPN.

Simple Memory System Example

❑ Addressing

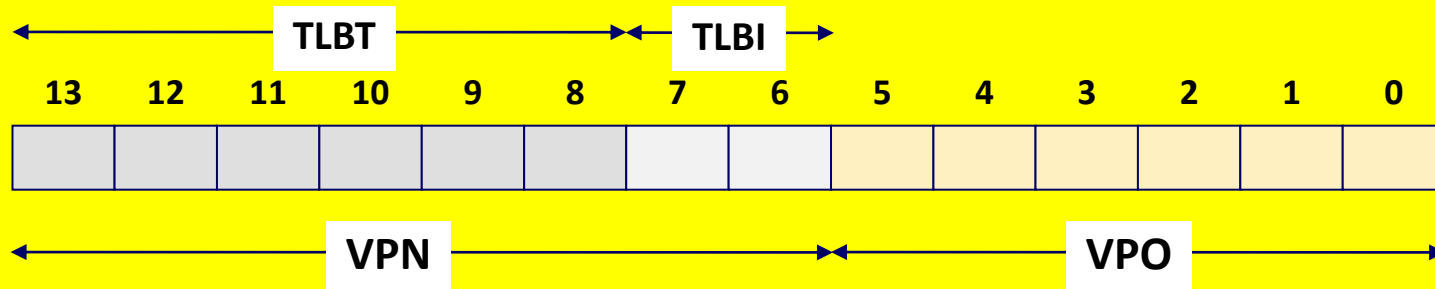
- The memory is byte addressable.
- Memory accesses are to 1-byte words (not 4-byte words).
- Virtual addresses are 14 bits wide ($n=14$) i.e., 14-bit virtual addresses
- Physical addresses are 12 bits wide ($m=12$) i.e., 12-bit physical address
- The page size is 64 bytes ($P=64$).



1. Simple Memory System TLB

Given : TLB is four-way set associative with 16 total entries.

- ★ 16 entries
- ★ 4-way associative



Set	Tag	PPN	Valid	Tag	PPN	Valid	Tag	PPN	Valid	Tag	PPN	Valid
0	03	–	0	09	0D	1	00	–	0	07	02	1
1	03	2D	1	02	–	0	04	–	0	0A	–	0
2	02	–	0	08	–	0	06	–	0	03	–	0
3	07	–	0	03	0D	1	0A	34	1	02	–	0

2. Simple Memory System Page Table

- Virtual addresses are 14 bits wide (n=14) i.e., 14-bit virtual addresses
- Physical addresses are 12 bits wide (m=12) i.e., 12-bit physical address
- The page size is 64 bytes (P =64).

Number of entries in Page Table = $2^{14} / 2^6 = 2^8 = 256$ page entries

Here we Only show first 16 entries (out of 256)

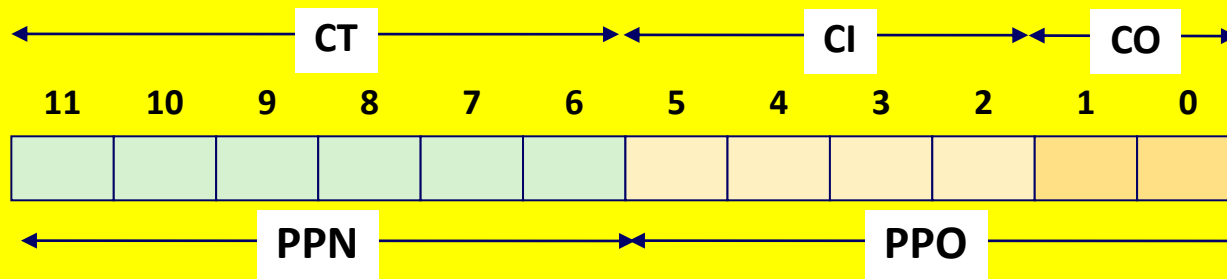
VPN	PPN	Valid
00	28	1
01	—	0
02	33	1
03	02	1
04	—	0
05	16	1
06	—	0
07	—	0

VPN	PPN	Valid
08	13	1
09	17	1
0A	09	1
0B	—	0
0C	—	0
0D	2D	1
0E	11	1
0F	0D	1

3. Simple Memory System Cache

The L1 d-cache is

- physically addressed and
- direct mapped,
- With a 4-byte line size and 16 total sets - 16 lines, 4-byte block size



<i>Idx</i>	<i>Tag</i>	<i>Valid</i>	<i>B0</i>	<i>B1</i>	<i>B2</i>	<i>B3</i>
0	19	1	99	11	23	11
1	15	0	–	–	–	–
2	1B	1	00	02	04	08
3	36	0	–	–	–	–
4	32	1	43	6D	8F	09
5	0D	1	36	72	F0	1D
6	31	0	–	–	–	–
7	16	1	11	C2	DF	03

<i>Idx</i>	<i>Tag</i>	<i>Valid</i>	<i>B0</i>	<i>B1</i>	<i>B2</i>	<i>B3</i>
8	24	1	3A	00	51	89
9	2D	0	–	–	–	–
A	2D	1	93	15	DA	3B
B	0B	0	–	–	–	–
C	12	0	–	–	–	–
D	16	1	04	96	34	15
E	13	1	83	77	1B	D3
F	14	0	–	– ⁴⁴	–	–

End-to-end Address Translation



□ TLB:

The TLB is virtually addressed using the bits of the VPN.

Since the TLB has four sets, the 2 low-order bits of the VPN serve as the set index (TLBI).

The remaining 6 high-order bits serve as the tag (TLBT) that distinguishes the different VPNs that might map to the same TLB set.

□ Page table

The page table is a single-level design with a total of $2^8=256$ page table entries (PTEs).

However, we are only interested in the first sixteen of these.

For convenience, we have labelled each PTE with the VPN that indexes it; but keep in mind that these VPNs are not part of the page table and not stored in memory.

Also, notice that the PPN of each invalid PTE is denoted with a dash to reinforce the idea that whatever bit values might happen to be stored there are not meaningful.

□ Cache

The direct-mapped cache is addressed by the fields in the physical address.

Since each block is 4bytes, the low-order 2bits of the physical address serve as the block offset(CO).

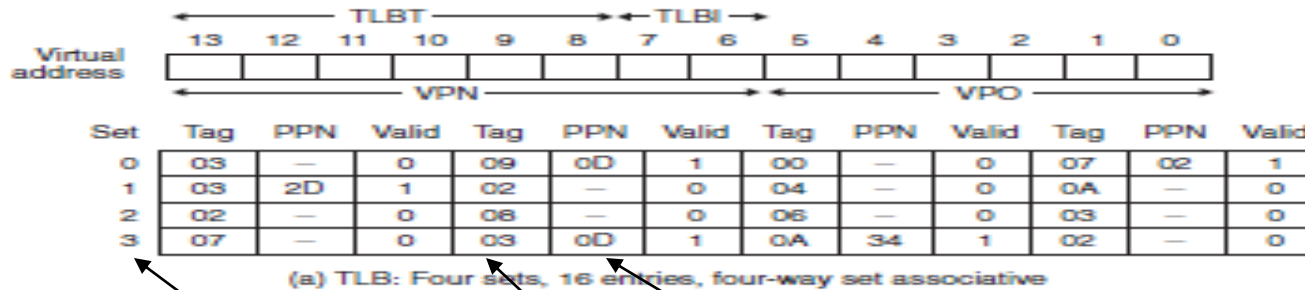
Since there are 16sets, the next 4bits serve as the set index (CI).

The remaining 6 bits serve as the tag (CT).

- Given this initial setup, let's see what happens when the CPU executes a load instruction that reads the byte at address 0x03d4. (Recall that our hypothetical CPU reads one-byte words rather than four-byte words.)
- To begin this kind of manual simulation, we find it helpful to write down the bits in the virtual address, identify the various fields we will need, and determine their hex values.
- The hardware performs a similar task when it decodes the address

End-to-end Address Translation

TLB, page table, and cache for small memory system. All values in the TLB, page table, and cache are in hexadecimal notation.

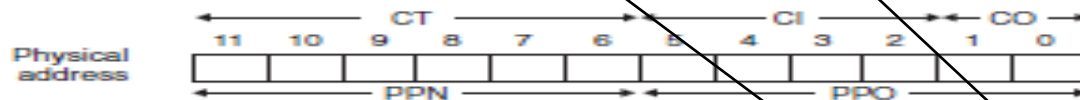


VA= 14 bits
VPO= 6 bits
VPN = 8 bits
TLBI= 2bits
TLBT= 6 bits

What happens when the CPU executes a load instruction that reads the byte at address 0x03D4.

VPN	PPN	Valid	VPN	PPN	Valid
00	2B	1	08	13	1
01	—	0	09	17	1
02	33	1	0A	09	1
03	02	1	0B	—	0
04	—	0	0C	—	0
05	16	1	0D	2D	1
06	—	0	0E	11	1
07	—	0	0F	0D	1

(b) Page table: Only the first 16 PTEs are shown

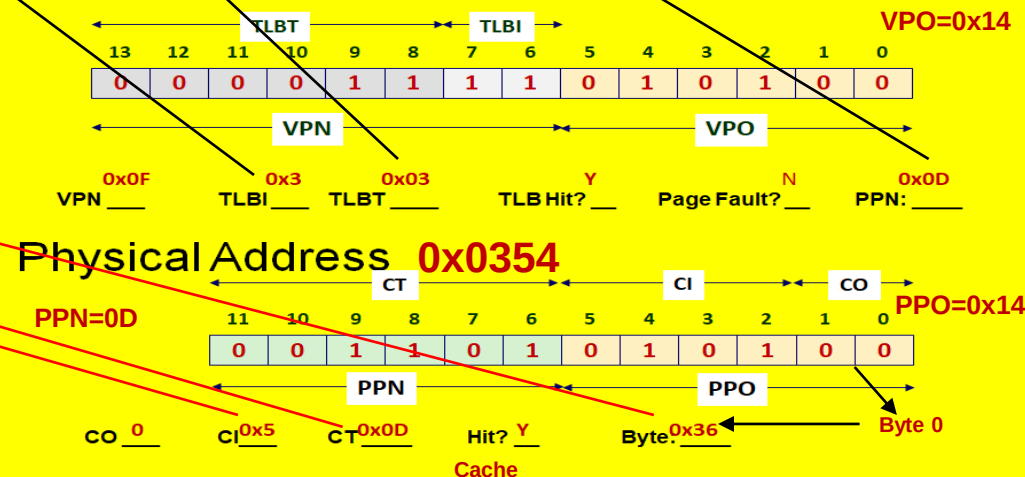


PA= 12 bits
PPO= 6 bits
PPN = 6 bits
CO= 2 bits
CI = 4 bits
CT= 6 bits

Idx	Tag	Valid	Blk 0	Blk 1	Blk 2	Blk 3
0	19	1	99	11	23	11
1	15	0	—	—	—	—
2	1B	1	00	02	04	0B
3	36	0	—	—	—	—
4	32	1	43	6D	8F	09
5	0D	1	36	72	F0	1D
6	31	0	—	—	—	—
7	16	1	41	C2	DF	03
8	24	1	3A	00	51	89
9	2D	0	—	—	—	—
A	2D	1	93	15	DA	3B
B	0B	0	—	—	—	—
C	12	0	—	—	—	—
D	16	1	04	96	34	15
E	13	1	83	77	1B	D3
F	14	0	—	—	—	—

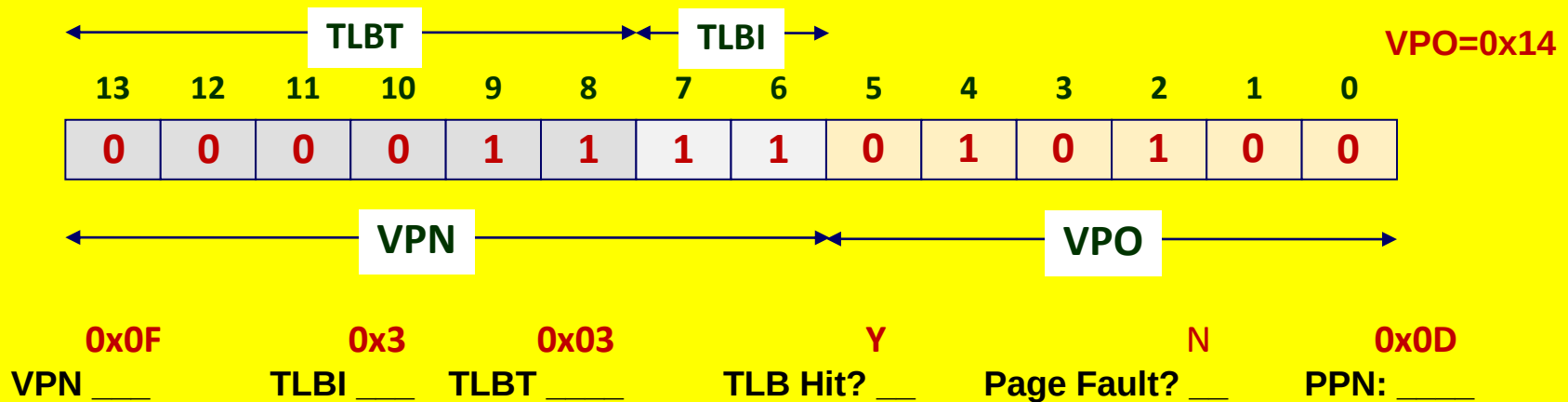
(c) Cache: Sixteen sets, 4-byte blocks, direct mapped

Virtual Address: 0x03D4

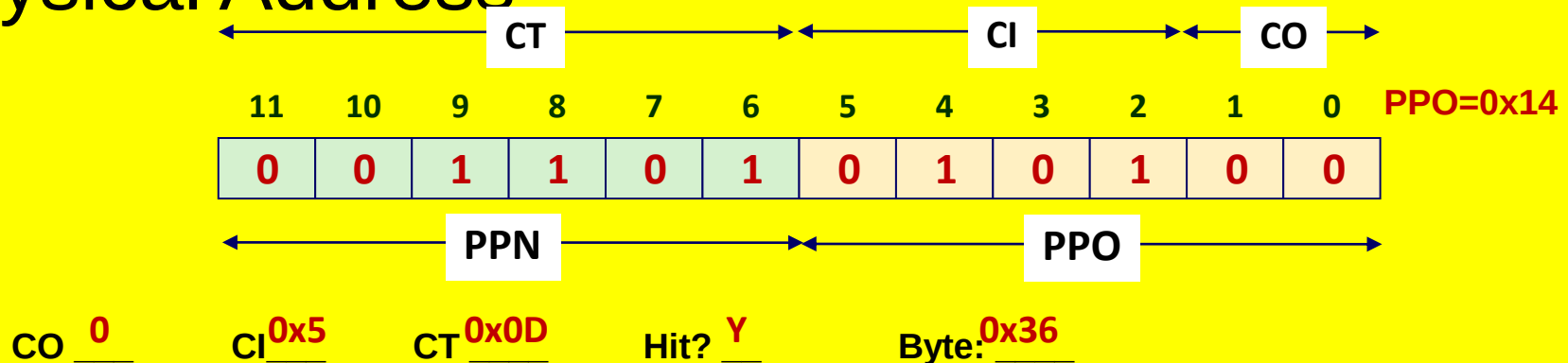


Address Translation Example #1

Virtual Address: 0x03D4

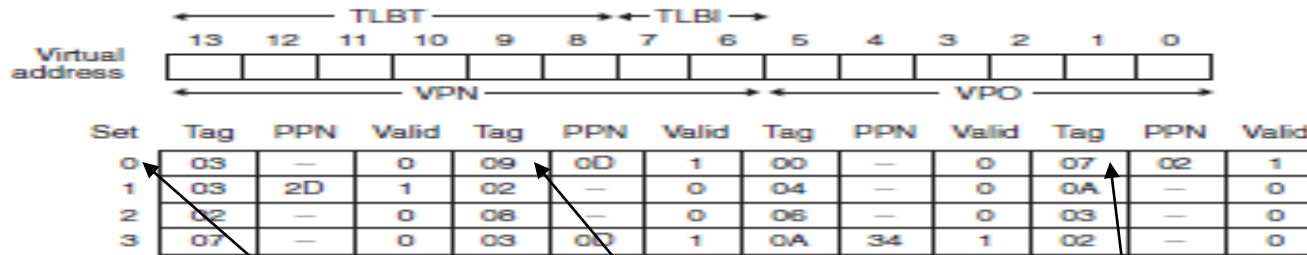


Physical Address 0x0354



End-to-end Address Translation

TLB, page table, and cache for small memory system. All values in the TLB, page table, and cache are in hexadecimal notation.



(a) TLB: Four sets, 16 entries, four-way set associative

VPN	PPN	Valid	VPN	PPN	Valid
00	2B	1	08	13	1
01	—	0	09	17	1
02	35	1	0A	09	1
03	02	1	0B	—	0
04	—	0	0C	—	0
05	16	1	0D	2D	1
06	—	0	0E	11	1
07	—	0	0F	0D	1

(b) Page table: Only the first 16 PTEs are shown



Idx	Tag	Valid	Blk 0	Blk 1	Blk 2	Blk 3
0	19	1	99	11	23	11
1	15	0	—	—	—	—
2	1B	1	00	02	04	0B
3	36	0	—	—	—	—
4	32	1	43	6D	8F	09
5	0D	1	36	72	F0	1D
6	31	0	—	—	—	—
7	16	1	11	C2	DF	03
8	24	1	3A	00	51	89
9	2D	0	—	—	—	—
A	2D	1	93	15	DA	3B
B	0B	0	—	—	—	—
C	12	0	—	—	—	—
D	16	1	04	96	34	15
E	13	1	83	77	1B	D3
F	14	0	—	—	—	—

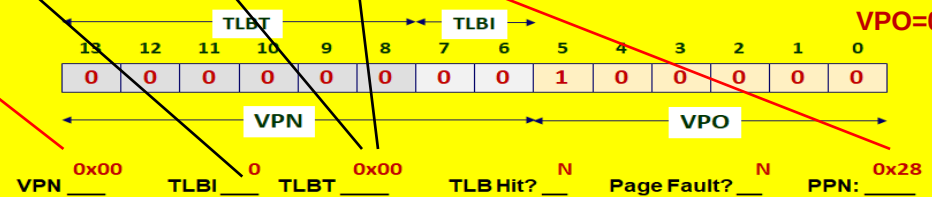
(c) Cache: Sixteen sets, 4-byte blocks, direct mapped

VA= 14 bits
VPO= 6 bits
VPN = 8 bits
TLBI= 2bits
TLBT= 6 bits

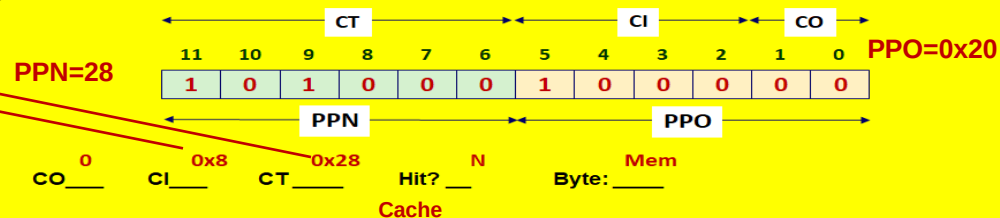
What happens when the CPU executes a load instruction that reads the byte at address 0x0020.

PA= 12 bits
PPO= 6 bits
PPN = 6 bits
CO= 2 bits
CI = 4 bits
CT= 6 bits

Virtual Address: 0x0020

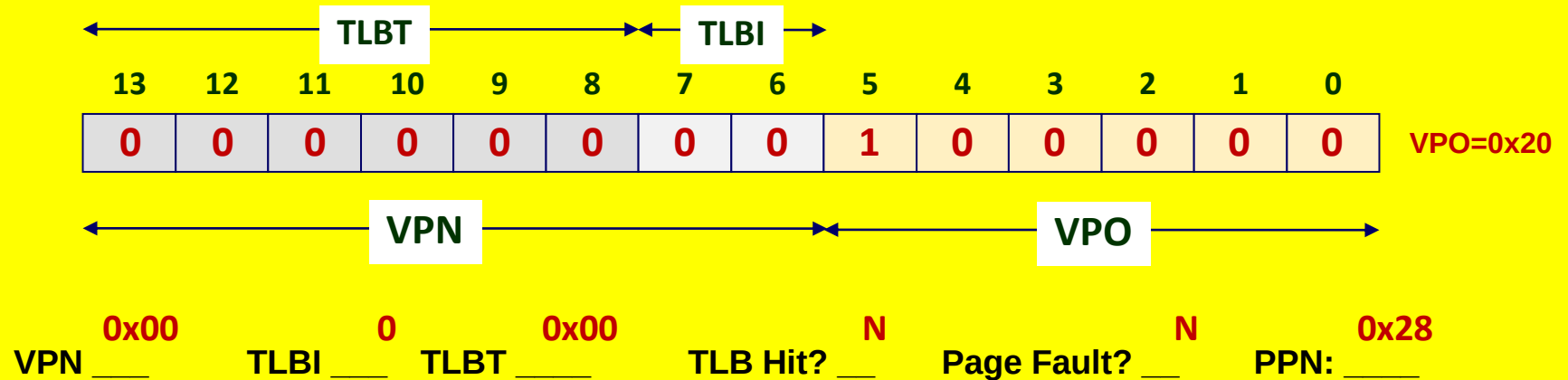


Physical Address 0x0A20

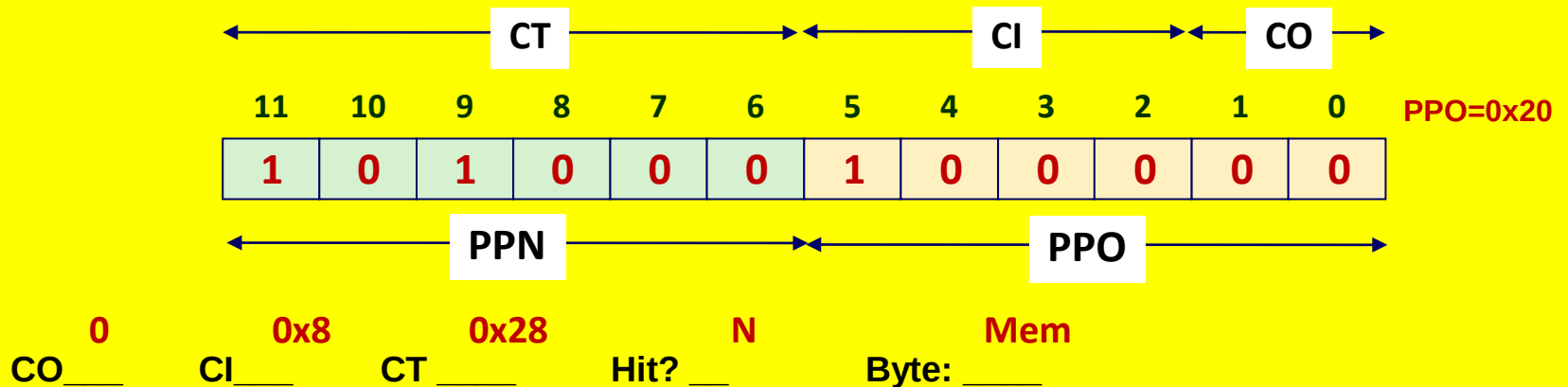


Address Translation Example #2

Virtual Address: 0x0020



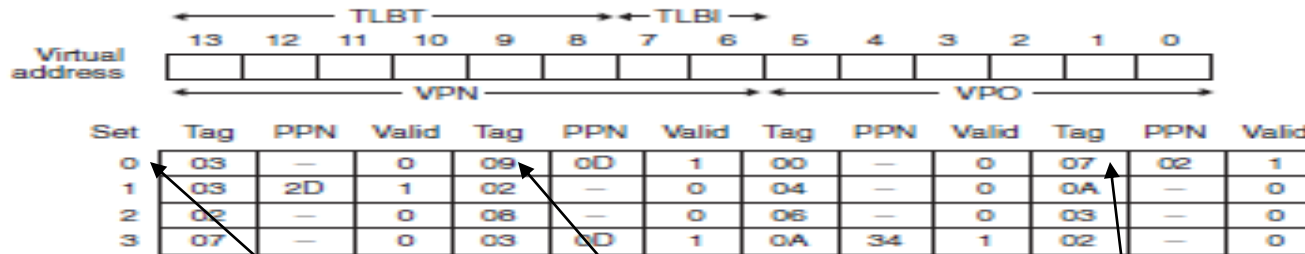
Physical Address 0x0A20



Cache Miss – required instruction is not in Cache - corresponding block needs to be brought from Main Memory

End-to-end Address Translation

TLB, page table, and cache for small memory system. All values in the TLB, page table, and cache are in hexadecimal notation.



(a) TLB: Four sets, 16 entries, four-way set associative

VPN	PPN	Valid	VPN	PPN	Valid
00	2B	1	08	13	1
01	—	0	09	17	1
02	33	1	0A	09	1
03	02	1	0B	—	0
04	—	0	0C	—	0
05	16	1	0D	2D	1
06	—	0	0E	11	1
07	—	0	0F	0D	1

(b) Page table: Only the first 16 PTEs are shown



Idx	Tag	Valid	Blk 0	Blk 1	Blk 2	Blk 3
0	19	1	99	11	23	11
1	15	0	—	—	—	—
2	1B	1	00	02	04	0B
3	36	0	—	—	—	—
4	32	1	43	6D	8F	09
5	0D	1	36	72	F0	1D
6	31	0	—	—	—	—
7	16	1	11	C2	DF	03
8	24	1	3A	00	51	89
9	2D	0	—	—	—	—
A	2D	1	93	15	DA	3B
B	0B	0	—	—	—	—
C	12	0	—	—	—	—
D	16	1	04	96	34	15
E	13	1	83	77	1B	D3
F	14	0	—	—	—	—

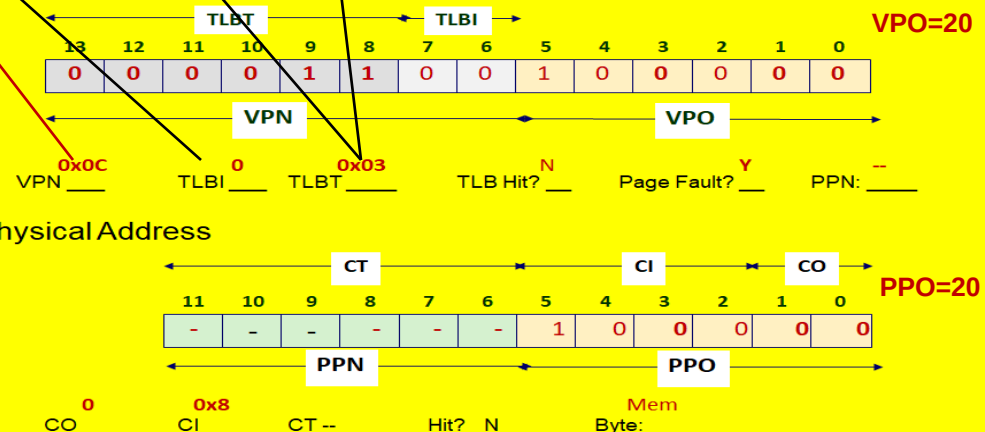
(c) Cache: Sixteen sets, 4-byte blocks, direct mapped

VA= 14 bits
VPO= 6 bits
VPN = 8 bits
TLBI= 2bits
TLBT= 6 bits

What happens when the CPU executes a load instruction that reads the byte at address 0x0320.

PA= 12 bits
PPO= 6 bits
PPN = 6 bits
CO= 2 bits
CI = 4 bits
CT= 6 bits

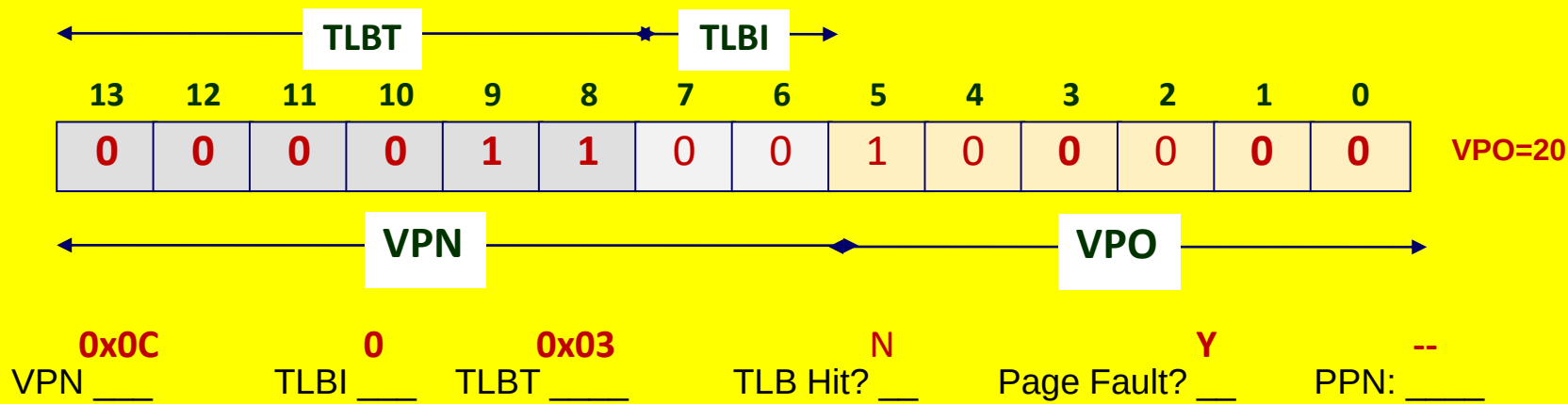
Virtual Address: 0x0320



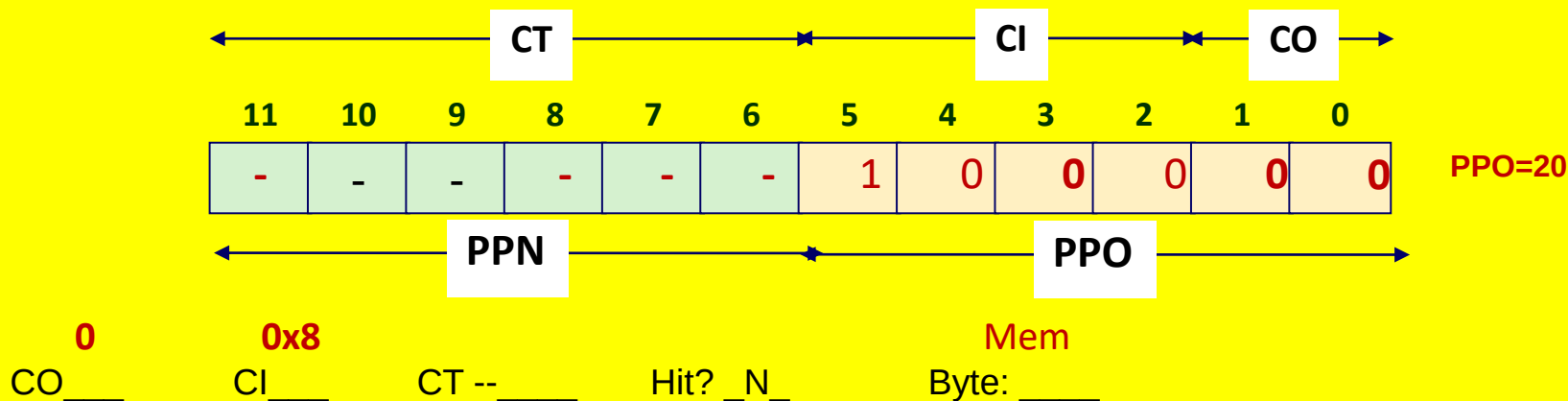
Page Fault – required Page is not in memory - needs to be brought from disk

Address Translation Example #3

Virtual Address: **0x0320**



Physical Address



Page Fault – required Page is not in memory - needs to be brought from disk

End-to-end Address Translation



- To begin, the MMU extracts the VPN (0x0F) from the virtual address and checks with the TLB to see if it has cached a copy of PTE 0x0F from some previous memory reference.
- The TLB extracts the TLB index (0x03) and the TLB tag (0x3) from the VPN, hits on a valid match in the second entry of Set 0x3, and returns the cached PPN (0x0D) to the MMU.
- If the TLB had missed, then the MMU would need to fetch the PTE from main memory.
- However, in this case we got lucky and had a TLBhit.
- The MMU now has everything it needs to form the physical address.
- It does this by concatenating the PPN (0x0D) from the PTE with the VPO (0x14) from the virtual address, which forms the physical address (0x354)

	TLBT								TLBI						
	0x03								0x03						
bit position	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
VA = 0x03d4	0	0	0	0	1	1	1	1	0	1	0	1	0	0	
	VPN								VPO						
	0x0f								0x14						



End-to-end Address Translation



- Next, the MMU sends the physical address to the cache, which extracts the cache offset CO (0x0), the cache set index CI (0x5), and the cache tag CT (0x0D) from the physical address.
- Since the tag in Set 0x5 matches CT, the cache detects a hit, reads out the data byte(0x36) at offset CO, and returns it to the MMU, which then passes it back to the CPU.
- Other paths through the translation process are also possible.
- For example, if the TLB misses, then the MMU must fetch the PPN from a PTE in the page table.
- If the resulting PTE is invalid, then there is a page fault and the kernel must page in the appropriate page and rerun the load instruction.
- Another possibility is that the PTE is valid, but the necessary memory block misses in the cache.

	CT						CI				CO	
	0x0d						0x05				0x0	
bit position	11	10	9	8	7	6	5	4	3	2	1	0
PA = 0x354	0	0	1	1	0	1	0	1	0	1	0	0
	PPN						PPO					
	0x0d						0x14				53	



Summary

- ★ **Programmer's view of virtual memory**
 - Each process has its own private linear address space
 - Cannot be corrupted by other processes
- ★ **System view of virtual memory**
 - Uses memory efficiently by caching virtual memory pages
 - ★ Efficient only because of locality
 - Simplifies memory management and programming
 - Simplifies protection by providing a convenient inter-positioning point to check permissions

VIRTUAL MEMORY SYSTEMS

LECTURE OUTLINE

- ★ **Case study:**
Core i7/Linux memory system
- ★ **Dynamic Memory Allocation**
- ★ **Memory Fragmentation**



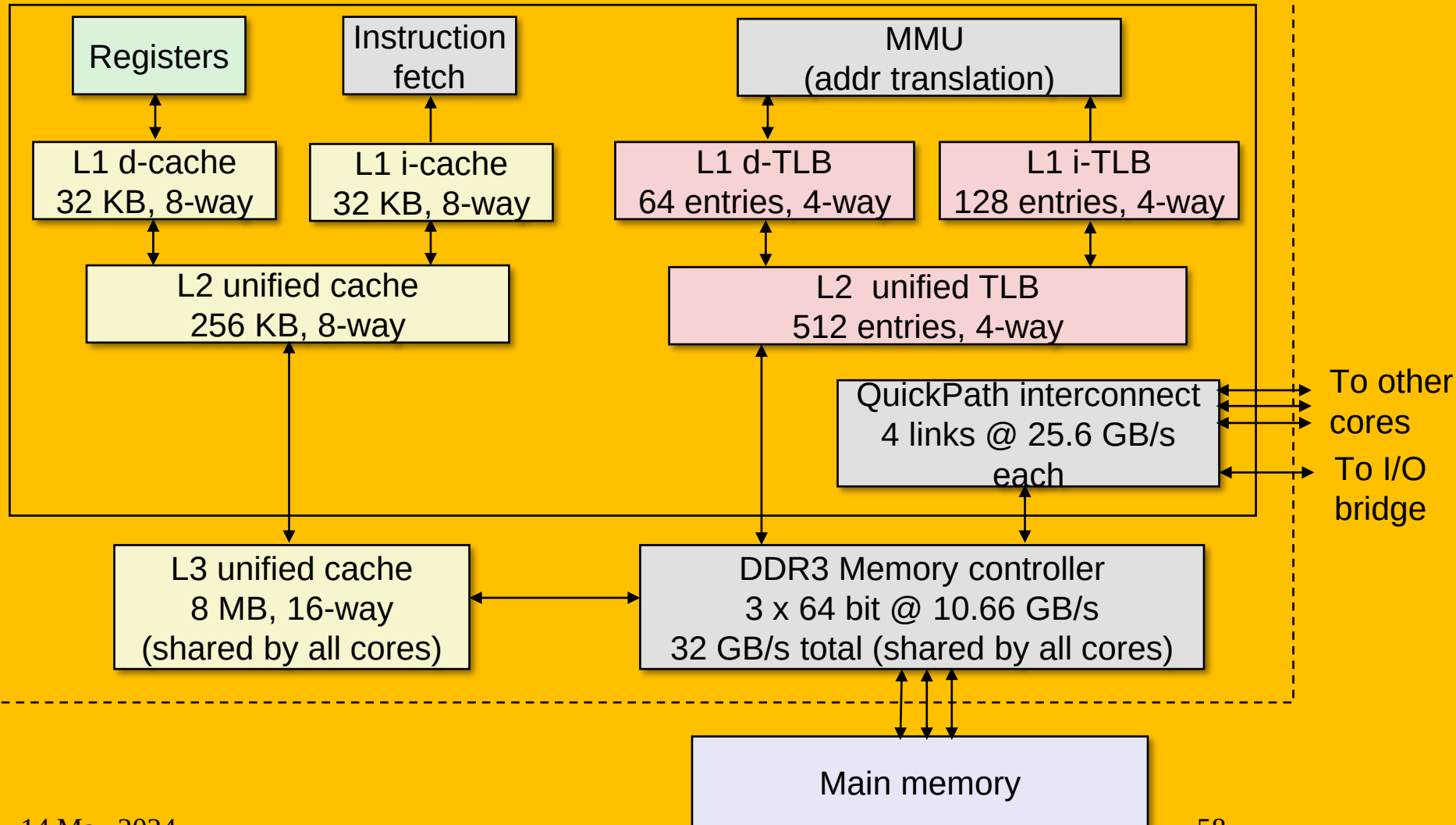
Case Study: The Intel Core i7/Linux Memory System

- ❑ **A case study of a real system :An Intel Corei7 running Linux**
- ★ **The Corei7 is based on the Nehalem microarchitecture.**
- ★ **Although the Nehalem design allows for full 64-bit virtual and physical address spaces, the current Core i7 implementations (and those for the foreseeable future) support a 48-bit (256 TB) virtual address space and a 52-bit (4 PB) physical address space, along with a compatibility mode that supports 32bit (4 GB) virtual and physical address spaces.**
- ★ **Figure gives the highlights of the Core i7 memory system.**
- ★ **The processor package includes four cores, a large L3 cache shared by all of the cores, and a DDR3 memory controller. Each core contains a hierarchy of TLBs, a hierarchy of data and instruction caches, and a set of fast point-to-point links, based on the Intel QuickPath technology, for communicating directly with the other cores and the external I/O bridge.**
- ★ **The TLBs are virtually addressed, and four-way set associative. The L1, L2, and L3 caches are physically addressed, and eight-way set associative, with a block size of 64 bytes.**
- ★ **The page size can be configured at start-up time as either 4 KB or 4 MB.**
- ★ **Linux uses 4-KB pages.**

Intel Core i7 Memory System

Processor package

Core x4



14 May 2024

58

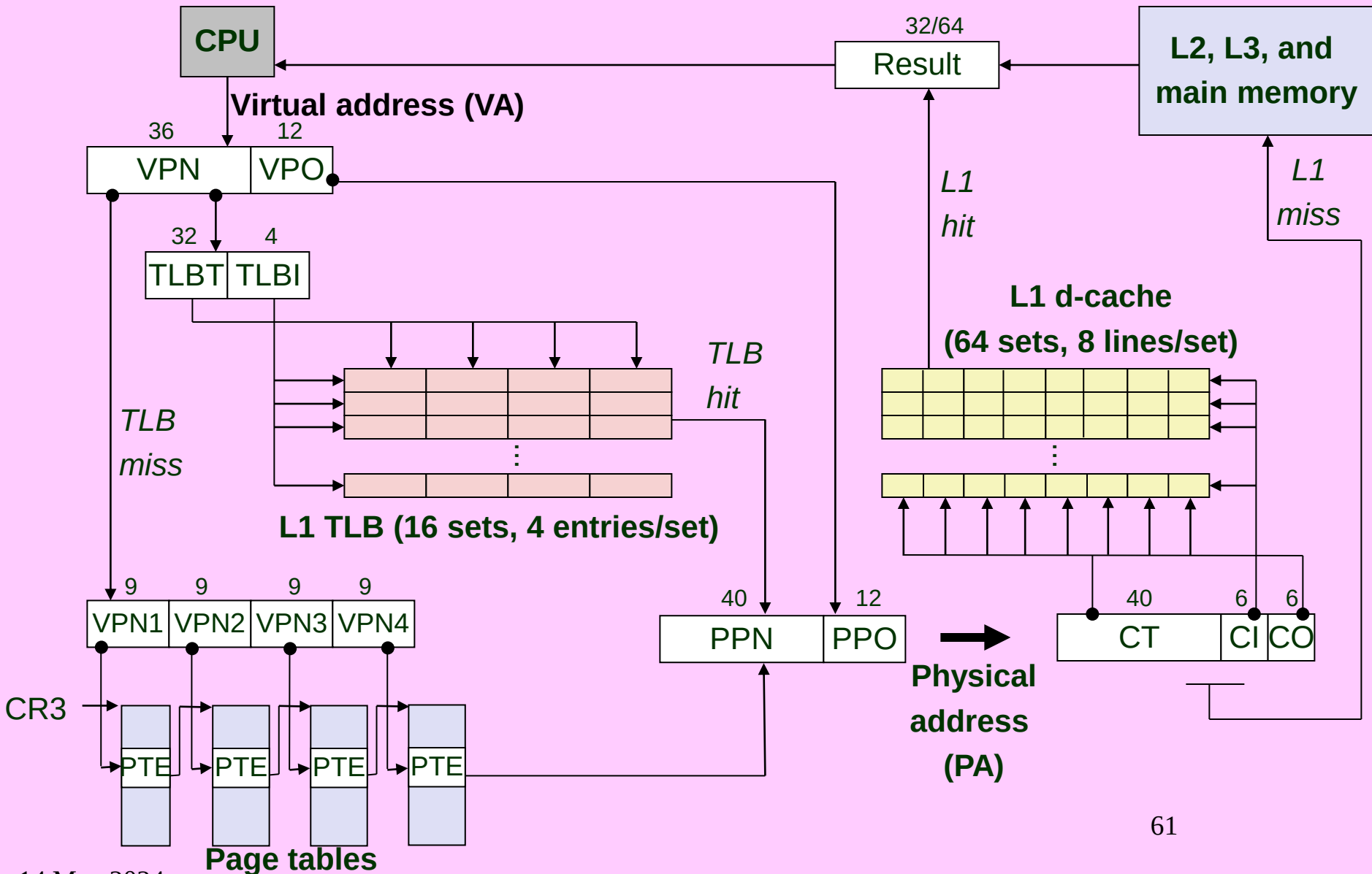
Core i7 Address Translation

- ★ **Figure summarizes the entire Core i7 address translation process, from the time the CPU generates a virtual address until a data word arrives from memory.**
- ★ **The Corei7 uses a four-level page table hierarchy.**
- ★ **Each process has its own private page table hierarchy.**
- ★ **When a Linux process is running, the page tables associated with allocated pages are all memory-resident, although the Core i7 architecture allows these page tables to be swapped in and out.**
- ★ **The CR3 control register points to the beginning of the level 1 (L1) page table.**
- ★ **The value of CR3 is part of each process context, and is restored during each context switch.**

Review of Symbols

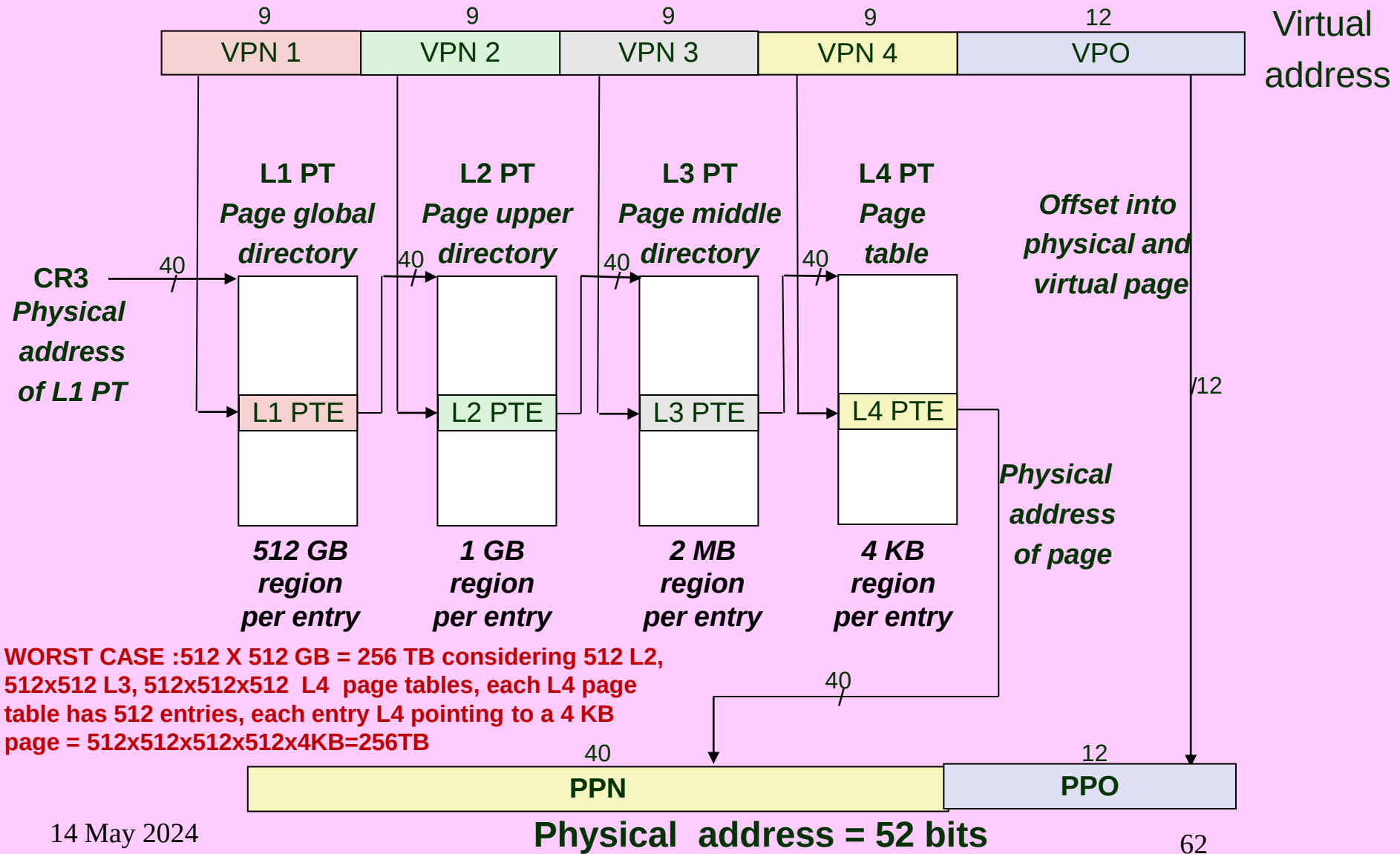
- ★ Basic Parameters
 - **N** = 2^n : Number of addresses in virtual address space
 - **M** = 2^m : Number of addresses in physical address space
 - **P** = 2^p : Page size (bytes)
- ★ Components of the virtual address (VA)
 - **TLBI**: TLB index
 - **TLBT**: TLB tag
 - **VPO**: Virtual page offset
 - **VPN**: Virtual page number
- ★ Components of the physical address (PA)
 - **PPO**: Physical page offset (same as VPO)
 - **PPN**: Physical page number
 - **CO**: Byte offset within cache line
 - **CI**: Cache index
 - **CT**: Cache tag

End-to-end Core i7 Address Translation

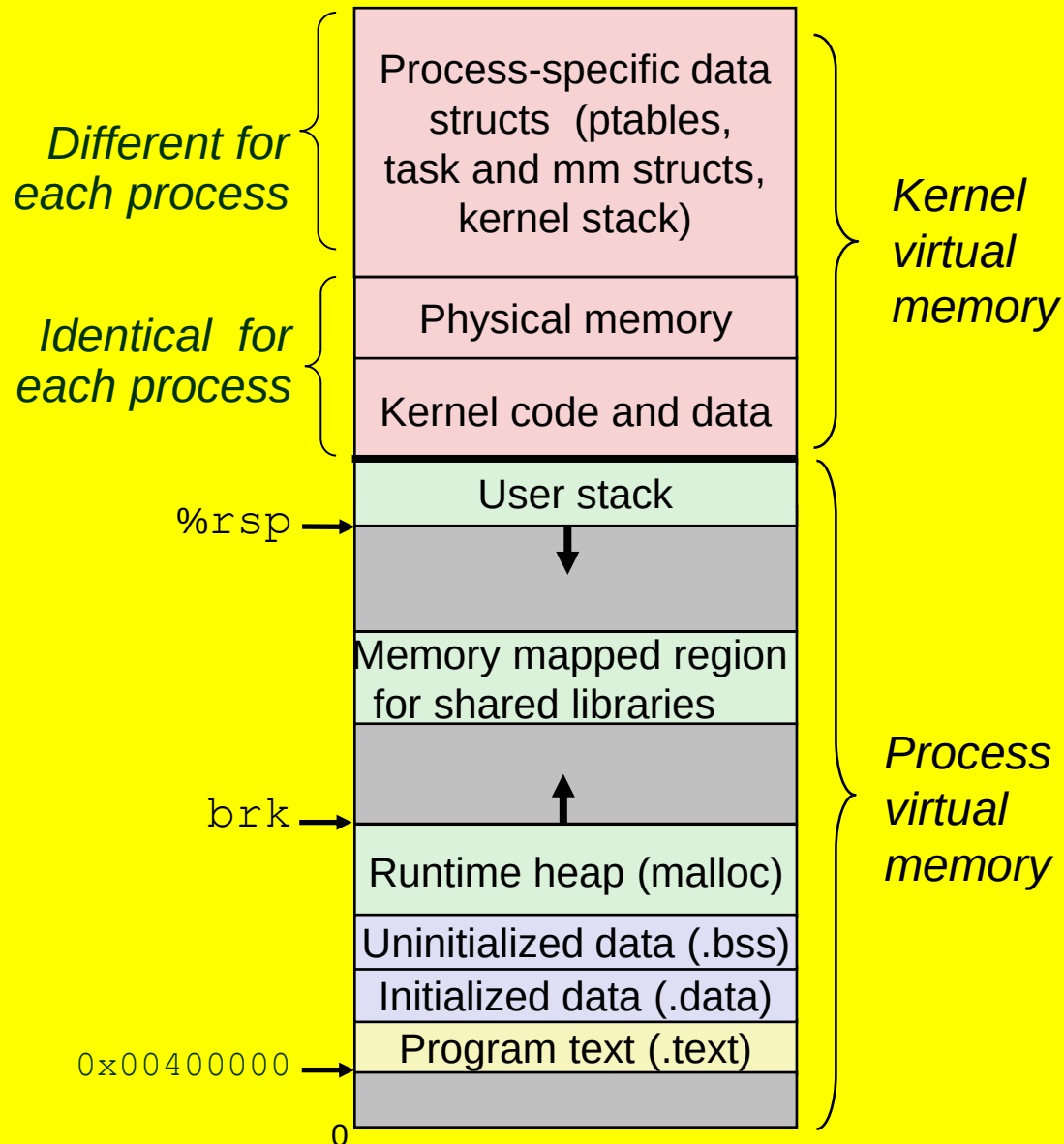


Core i7 Page Table Translation

Virtual address = 48 bits



Virtual Address Space of a Linux Process



DYNAMIC MEMORY ALLOCATION : BASIC CONCEPTS

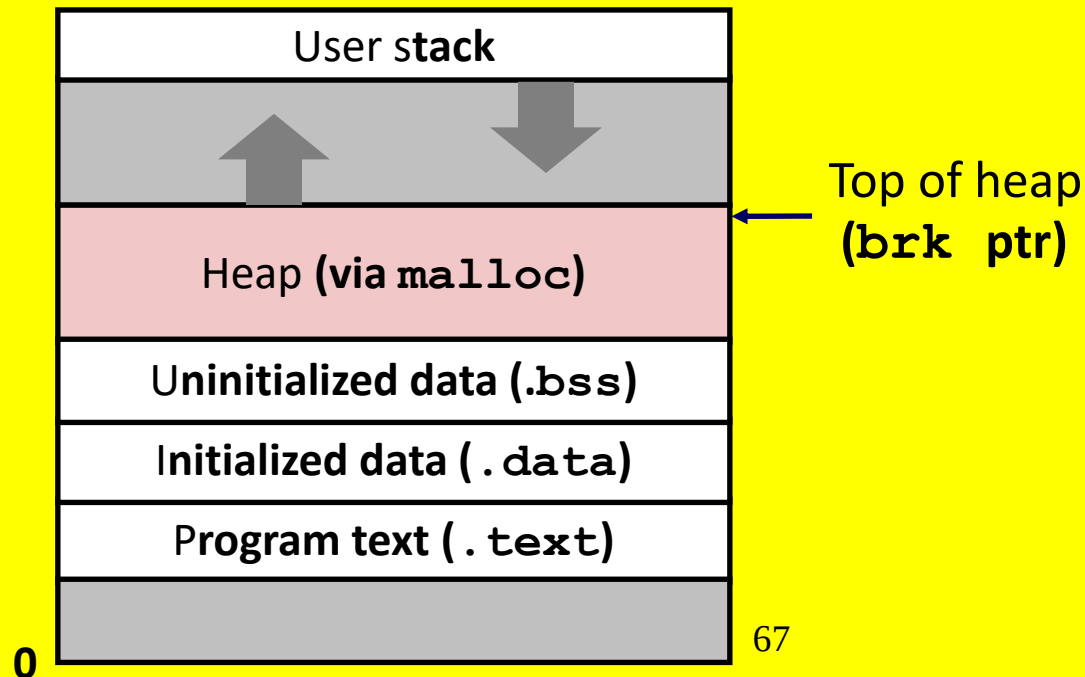
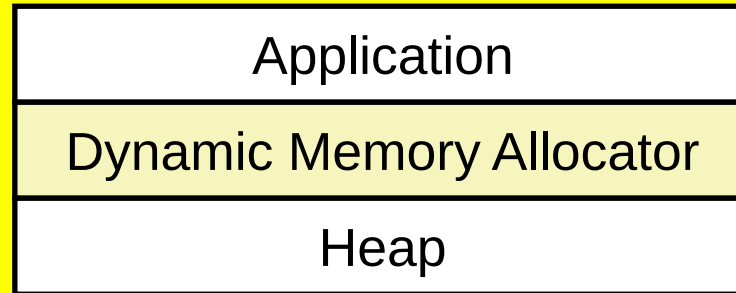
DYNAMIC MEMORY ALLOCATION : BASIC CONCEPTS

Dynamic Memory Allocation

- ★ While it is certainly possible to use the low-level **mmap and munmap functions** to create and delete areas of virtual memory, C programmers typically find it more convenient and more portable to use a dynamic memory allocator when they need to acquire additional virtual memory at run time.
- ★ A dynamic memory allocator maintains an area of a process's virtual memory known as the heap.
- ★ Details vary from system to system, but without loss of generality, we will assume that the **heap is an area of demand-zero memory that begins immediately after the uninitialized bss area and grows upward (toward higher addresses).**

Dynamic Memory Allocation

- ★ Programmers use *dynamic memory allocators* (such as `malloc`) to acquire VM at run time.
 - For data structures whose size is only known at runtime.
- ★ Dynamic memory allocators manage an area of process virtual memory known as the *heap*.



Dynamic Memory Allocation

- ★ For each process, the kernel maintains a variable `brk` (pronounced “break”) that points to the top of the heap.
- ★ An allocator maintains the heap as a collection of various-sized blocks.
- ★ Each block is a contiguous chunk of virtual memory that is either allocated or free.
- ★ An allocated block has been explicitly reserved for use by the application.
- ★ A free block is available to be allocated.
- ★ A free block remains free until it is explicitly allocated by the application.
- ★ An allocated block remains allocated until it is freed, either explicitly by the application, or implicitly by the memory allocator itself.

Dynamic Memory Allocation

- ❑ **Allocators come in two basic styles.**
 - ★ **Explicit allocators**
 - ★ **Implicit allocators**
- ★ **Both styles require the application to explicitly allocate blocks.**
- ★ **They differ about which entity is responsible for freeing allocated blocks.**

Dynamic Memory Allocation

- ★ Allocator maintains heap as collection of variable sized *blocks*, which are either *allocated* or *free*
- ★ Types of allocators
 - *Explicit allocator*: application allocates and frees space
 - ★ E.g., `malloc` and `free` in C
 - *Implicit allocator*: application allocates, but does not free space
 - ★ E.g. garbage collection in Java, ML, and Lisp

Dynamic Memory Allocation

❑ Explicit allocators

- ★ require the application to explicitly free any allocated blocks.
- ★ For example, the C standard library provides an explicit allocator called the malloc package.
- ★ C programs allocate a block by calling the malloc function, and free a block by calling the free function.
- ★ The new and delete calls in C++ are comparable

❑ Implicit allocators

- ★ on the other hand, require the allocator to detect when an allocated block is no longer being used by the program and then free the block.
- ★ Implicit allocators are also known as garbage collectors, and the process of automatically freeing unused allocated blocks is known as garbage collection.
- ★ For example, higher-level languages such as Lisp, ML, and Java rely on garbage collection to free allocated blocks.

malloc and free Functions

- ★ The C standard library provides an explicit allocator known as the malloc package.
- ★ Programs allocate blocks from the heap by calling the malloc function.

```
#include <stdlib.h>
```

```
void *malloc(size_t size);
```

Returns: ptr to allocated block if OK, NULL on error

- ★ The malloc function returns a pointer to a block of memory of atleast size bytes that is suitably aligned for any kind of data object that might be contained in the block.

malloc and free Functions

- ★ On the Unix systems that we are familiar with, **malloc** returns a block that is aligned to an 8-byte (double word) boundary.
- ★ If **malloc** encounters a problem (e.g., the program requests a block of memory that is larger than the available virtual memory), then it returns NULL and sets **errno**.
- ★ **Malloc** does not initialize the memory it returns.
- ★ Applications that want initialized dynamic memory can use **calloc**, a thin wrapper around the **malloc** function that initializes the allocated memory to zero.
- ★ Applications that want to change the size of a previously allocated block can use the **realloc** function.

Dynamic Memory Allocation

- ★ Dynamic memory allocators such as malloc can allocate or deallocate heap memory explicitly by using the mmap and munmap functions, or they can use the sbrk function:

```
#include <unistd.h>
```

```
void *sbrk(intptr_t incr);
```

Returns: old brk pointer on success, -1 on error

- ★ The sbrk function grows or shrinks the heap by adding incr to the kernel's brk pointer.
- ★ If successful, it returns the old value of brk, otherwise it returns -1 and sets errno to ENOMEM. If incr is zero, then sbrk returns the current value of brk.
- ★ Calling sbrk with a negative incr is legal but tricky because the return value (the old value of brk) points to abs(incr) bytes past the new top of the heap.
- ★ Programs free allocated heap blocks by calling the free function.

Dynamic Memory Allocation

- ★ Programs free allocated heap blocks by calling the free function.

```
#include <stdlib.h>  
void free(void *ptr);  
Returns: nothing
```

- ★ The ptr argument must point to the beginning of an allocated block that was obtained from malloc, calloc, or realloc.
- ★ If not, then the behavior of free is undefined.
- ★ Even worse, since it returns nothing, free gives no indication to the application that something is wrong.
- ★ This can produce some baffling run-time errors.

The malloc Package

```
#include <stdlib.h>
```

```
void *malloc(size_t size)
```

- **Successful:**
- Returns a pointer to a memory block of at least `size` bytes aligned to an 8-byte (x86) or 16-byte (x86-64) boundary
 - * If `size == 0`, returns `NULL`
- **Unsuccessful:** returns `NULL (0)` and sets `errno`

```
void free(void *p)
```

- Returns the block pointed at by `p` to pool of available memory
- `p` must come from a previous call to `malloc` or `realloc`

Other functions

- `calloc`: Version of `malloc` that initializes allocated block to zero.
- `realloc`: Changes the size of a previously allocated block.
- `sbrk`: Used internally by allocators to grow or shrink the heap

Dynamic Memory Allocation

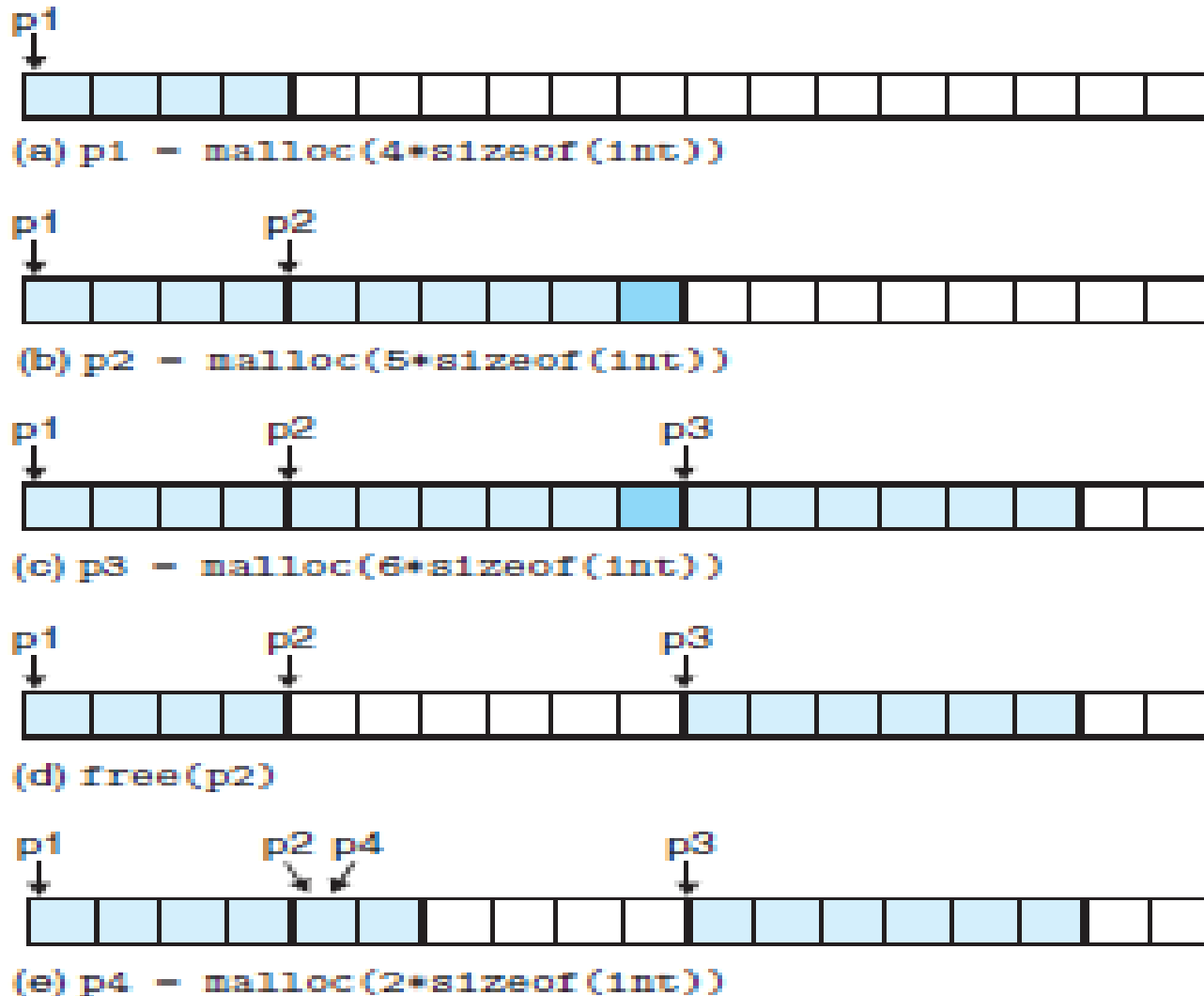


Figure - Allocating and freeing blocks with malloc and free.

Each square corresponds to a word. Each heavy rectangle corresponds to a block. Allocated blocks are shaded. Padded regions of allocated blocks are shaded with stripes. Free blocks are unshaded. Heap addresses increase from left to right.

Dynamic Memory Allocation



Figure shows how an implementation of malloc and free might manage a (very) small heap of 16 words for a C program.

Each box represents a 4-byte word.

The heavy-lined rectangles correspond to allocated blocks (shaded) and free blocks (unshaded).

Initially, the heap consists of a single 16-word doubleword aligned free block.

In figure The program asks for a four-word block.

Malloc responds by carving out a four-word block from the front of the free block and returning a pointer to the first word of the block. .

Next in figure the program requests a five-word block.

Malloc responds by allocating a six-word block from the front of the free block.

In this example, malloc pads the block with an extra word in order to keep the free block aligned on a double-word boundary. .

Next in Figure : The program requests a six-word block and malloc responds by carving out a six-word block from the free block. .

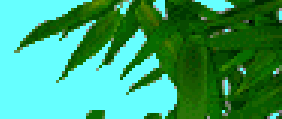
Lastly in Figure : The program frees the six-word block that was earlier allocated.

Notice that after the call to free returns, the pointer p2 still points to the freed block. It is the responsibility of the application not to use p2 again until it is reinitialized by a new call to malloc.

Fragmentation

- ★ The primary cause of poor heap utilization is a phenomenon known as **fragmentation**, which occurs when otherwise unused memory is not available to satisfy allocate requests.
- ★ There are two forms of fragmentation:
 - ***Internal*** fragmentation
 - ***External*** fragmentation

Fragmentation

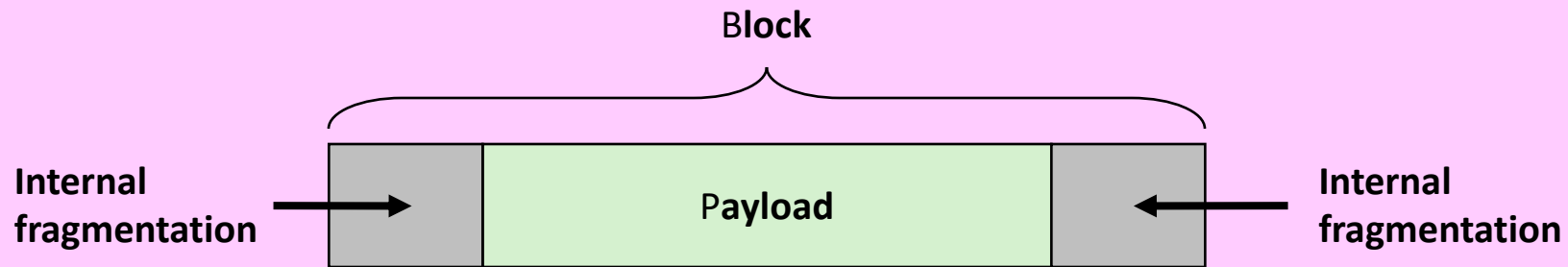


❑ Internal Fragmentation

- ★ Internal fragmentation occurs when an allocated block is larger than the payload.
- ★ This might happen for a number of reasons.
- ★ For example, the implementation of an allocator might impose a minimum size on allocated blocks that is greater than some requested payload.
- ★ Or, the allocator might increase the block size in order to satisfy alignment constraints.
- ★ Internal fragmentation is straightforward to quantify.
- ★ It is simply the sum of the differences between the sizes of the allocated blocks and their payloads.
- ★ Thus, at any point in time, the amount of internal fragmentation depends only on the pattern of previous requests and the allocator implementation.

Internal Fragmentation

- ★ For a given block, *internal fragmentation* occurs if payload is smaller than block size



- ★ Caused by
 - Overhead of maintaining heap data structures
 - Padding for alignment purposes
 - Explicit policy decisions (e.g., to return a big block to satisfy a small request)
- ★ Depends only on the pattern of *previous* requests
 - Thus, easy to measure

External Fragmentation

- External fragmentation occurs when there is enough aggregate free memory to satisfy an allocate request, but no single free block is large enough to handle the request.



(a) `p1 = malloc(4*sizeof(int))`



(b) `p2 = malloc(5*sizeof(int))`



(c) `p3 = malloc(6*sizeof(int))`



(d) `free(p2)`



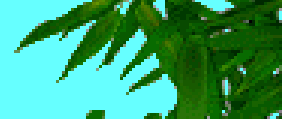
(e) `p4 = malloc(2*sizeof(int))`

(f) `p5 = malloc(6)`

Oops! (what would happen now?)

- For example, if the next request p5 in Figure after (e) is for six words, then the request cannot be satisfied without requesting additional virtual memory from the kernel, even though there are six free words remaining in the heap. The problem arises because these six words are spread over two free blocks.

Fragmentation



❑ External Fragmentation

- ★ External fragmentation occurs when there is enough aggregate free memory to satisfy an allocate request, but no single free block is large enough to handle the request.
- ★ External fragmentation is much more difficult to quantify than internal fragmentation because it depends not only on the pattern of previous requests and the allocate or implementation, but also on the pattern of future requests.
- ★ For example, suppose that after k requests all of the free blocks are exactly four words in size.
- ★ Does this heap suffer from external fragmentation?
- ★ The answer depends on the pattern of future requests.
- ★ If all of the future allocate requests are for blocks that are smaller than or equal to four words, then there is no external fragmentation.
- ★ On the other hand, if one or more requests ask for blocks larger than four words, then the heap does suffer from external fragmentation.
- ★ Since external fragmentation is difficult to quantify and impossible to predict, allocators typically employ heuristics that attempt to maintain small numbers of larger free blocks rather than large numbers of smaller free blocks.

SOC 2040 SYSTEM PROGRAMMING

Reading Assignments

Chapter 9 of Text Book

➤ Computer Systems : A Programmer's Perspective

- Randal E. Bryant and David R. O'Hallaron
3rd Edition, Global Edition Prentice Hall 2016
ISBN 10:1-292-10176-8**

SOC 2040

SYSTEM PROGRAMMING



END OF

WEEK 14 Lecture 2

VIRTUAL MEMORY

WISH YOU ALL THE BEST

14 May 2024

SPRING Semester 2024 @ Dr A R Naseer





INHA UNIVERSITY TASHKENT

SPRING SEMESTER 2024

SOC 2040

SYSTEM PROGRAMMING

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR

DR. A. R. NASEER

14 May 2024

HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG